Web aplikacije (WA - 199769)

Nositelj: doc. dr. sc. Nikola Tanković
Asistent: Luka Blašković, mag. inf.
Ustanova: Sveučilište Jurja Dobrile u Puli, Fakultet informatike u Puli

Fakultet informatike u Puli

(3) Razmjena podataka između klijenta i poslužitelja

Do sada smo obradili izradu Express poslužitelja koji može isporučivati statične datoteke i JSON podatke klijentskoj strani, pri čemu nam je Postman služio kao osnovno klijentsko okruženje. Naučili smo kako validirati i obraditi podatke koji pristižu na poslužitelj putem različitih HTTP metoda (GET, POST, PUT, PATCH, DELETE).

U ovoj skripti prelazimo na razvoj klijentskog dijela web aplikacije, koji ćemo izraditi koristeći Vue.js razvojni okvir. Budući da se Vue.js detaljno obrađuje u sklopu kolegija Programsko inženjerstvo, ovdje se nećemo zadržavati na njegovim specifičnostima. Umjesto toga, usredotočit ćemo se na praktičnu realizaciju komunikacije između klijentske i poslužiteljske strane.

Preporučuje se da prije početka izrade *frontend* dijela web aplikacije pripremite stabilan Express.js poslužitelj na temelju zadnje dvije skripte te ga temeljito testirate u Postmanu. U ovoj skripti ćemo implementirati jednostavnu Vue.js aplikaciju za naručivanje Pizze koja će komunicirati Express poslužiteljem, a nakon toga preporuka je paralelno razvijati i testirati klijentski i poslužiteljski sloj aplikacije.

🆙 **Posljednje ažurirano: 12.9.2026.**

Sadržaj

- Web aplikacije (WA - 199769)
- (3) Razmjena podataka između klijenta i poslužitelja

- Sadržaj
- 1. Postavljanje Express poslužitelja
  - 1.1 Definiranje osnovnih endpointova i dummy podataka
    - 1.1.1 Implementacija `/pizze Router`
    - 1.1.2 Implementacija `/narudzbe Router`
- 2. Implementacija Vue.js klijentske strane
  - 2.1 Konfiguracija Vue.js projekta s TailwindCSS-om i Vite-om
  - 2.2 TailwindCSS konfiguracija
  - 2.3 Dodavanje osnovnih komponenti korisničkog sučelja
    - PizzaList.vue komponenta
    - Header.vue komponenta
    - Implementacija odabira pizze
- 3. Axios i komunikacija s Express poslužiteljem
  - 3.1 CORS politika
  - 3.2 Dinamičko iscrtavanje podataka o pizzama (GET /pizze)
    - 3.2.1 `v-for` direktiva
    - 3.2.2 Prikaz ikona sastojaka
    - 3.2.3 Dodavanje javnih slika na poslužitelj
  - 3.3 Slanje nove narudžbe (POST /narudzbe)
    - Opcionalno lančanje (Optional Chaining)
    - Emitiranje događaja (Event Emitting)
    - Implementacija preostalih UI funkcionalnosti
- Samostalni zadatak za Vježbu 3

1. Postavljanje Express poslužitelja

Krenimo s definiranjem osnovnog `Express` poslužitelja koji će služiti kao *backend* za našu aplikaciju za naručivanje pizze. Možete ponovno iskoristiti kod iz prethodnih skript ili započeti od nule (u tom slučaju preskočite na poglavlje 2).

Definirat ćemo dva direktorija naše web aplikacije (`pizza-express` za poslužitelj i `pizza-vue` za klijenta):

```
app
├── pizza-express
└── pizza-vue
```

Prebacite se u direktorij `pizza-express` i inicijalizirajte osnovni `Express.js` projekt:

```
→ cd app/pizza-express
→ npm init -y
→ npm install express
```

```
// app/pizza-express/index.js

import express from 'express';

const app = express();
const PORT = 3000;
app.use(express.json());

app.get('/', (req, res) => {
    res.send('Dobrodošli u Pizza Express poslužitelj!');
});

app.listen(PORT, () => {
    console.log(`Pizza poslužitelj sluša na portu ${PORT}`);
});
```

```
→ nodemon index.js

Pizza poslužitelj sluša na portu 3000
```

Dodajte u `package.json` datoteku `"type": "module"` kako biste omogućili korištenje ES modula, te isključite verzioniranje `node_modules` direktorija dodavanjem `.gitignore` datoteke:

```
→ echo "node_modules/" > .gitignore
```

```
// .gitignore
node_modules/
```

1.1 Definiranje osnovnih endpointova i dummy podataka

Dodajmo osnovne *endpointove* za upravljanje narudžbama pizza. Implementirat ćemo sljedeće rute:

- `GET /pizze` - Dohvaćanje dostupnih pizza
- `GET /pizze/:naziv` - Dohvaćanje detalja o određenoj pizzi
- `POST /narudzbe` - Izrada nove narudžbe pizza

Stvorite `Router` objekte za pizze i narudžbe:

```
→ mkdir routes
→ touch routes/pizze.js routes/narudzbe.js
```

Dodat ćemo i nešto *in-memory dummy* podataka za pizze i narudžbe unutar `data/data.js` datoteke:

```
→ mkdir data
→ touch data/data.js
```

Primjer podataka iz `data/data.js` datoteke:

```
// app/pizza-express/data/data.js

export const pizze = [
    { id: 1, naziv: 'Margherita', sastojci: ['rajčica', 'sir', 'bosiljak'], cijene: { mala: 7.3, srednja: 9.2, jumbo: 16.2 } },
    { id: 2, naziv: 'Capricciosa', sastojci: ['rajčica', 'sir', 'šunka', 'gljive'], cijene: { mala: 7.9, srednja: 9.9, jumbo: 18.0 } },
    { id: 3, naziv: 'Al Tonno', sastojci: ['rajčica', 'sir', 'tunjevina', 'crveni luk'], cijene: { mala: 8.7, srednja: 11.7, jumbo: 21.9 } },
    { id: 4, naziv: 'Fantasia', sastojci: ['rajčica', 'sir', 'gljive', 'šunka', 'paprika', 'panceta', 'vrhnje'], cijene: { mala: 9.4, srednja: 12.7, jumbo: 22.2 } },
    { id: 5, naziv: 'Slavonska', sastojci: ['rajčica', 'sir', 'kulen', 'panceta', 'feferoni ljuti', 'paprika', 'crveni luk'], cijene: { mala: 9.9, srednja: 13.2, jumbo: 22.9 } }
];

export const narudzbe = [
    {
        id: 1,
        narucene_pizze: [
            // svaka narudžba sastoji se od jedne ili više naručenih pizza
            {
                // za svaku naručenu pizzu bilježimo naziv, naručenu veličinu i količinu
                naziv: 'Margherita',
                velicina: 'srednja',
                kolicina: 2
            },
            {
                naziv: 'Fantasia',
                velicina: 'jumbo',
                kolicina: 1
            }
        ],
        ukupna_cijena: 40.6, // ukupnu cijenu narudžbe računamo na poslužitelju
        podaci_dostava: {
            prezime: 'Perić',
            adresa: 'Zagrebačka 15, Pula',
            telefon: '091234567'
        }
    }
];
```

Struktura Express.js projekta sada nam izgleda ovako:

```
app
└── pizza-express
    ├── node_modules
    ├── index.js
    ├── package.json
    ├── package-lock.json
    ├── data
    │   └── data.js
    ├── routes
    │   ├── pizze.js
    │   └── narudzbe.js
    └── .gitignore
└── pizza-vue
```

1.1.1 Implementacija `/pizze Router`

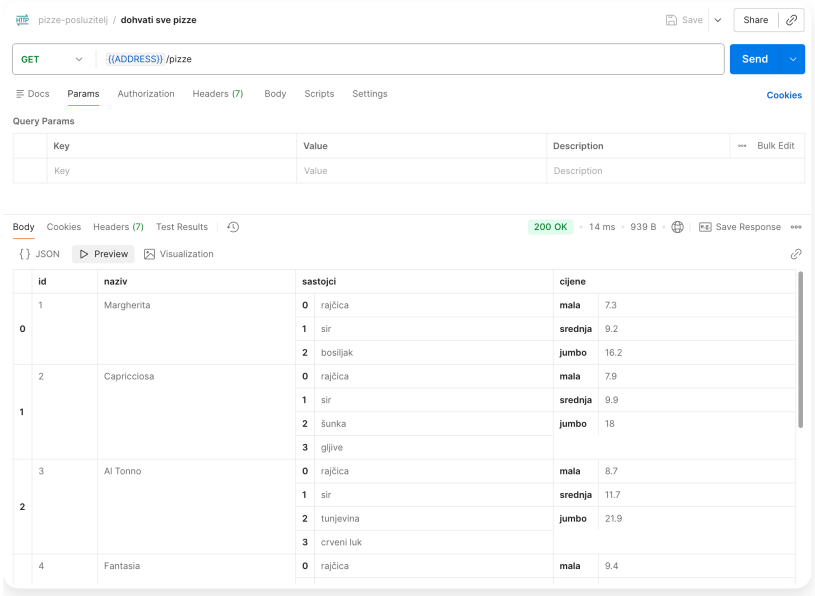
U ovaj `Router` dodajemo rute za dohvaćanje svih pizza i pojedinačne pizze prema nazivu.

```
// app/pizza-express/routes/pizze.js

import express from 'express';
import { pizze } from '../data/data.js'; // učitavanje dummy podataka
const router = express.Router();

// GET /pizze - Dohvaćanje svih pizza (npr. GET /pizze)
router.get('/', (req, res) => {
    if (pizze.length === 0 || !pizze) {
        return res.status(404).json({ message: 'Nema dostupnih pizza.' });
    }

    res.status(200).json(pizze);
});

// GET /pizze/:naziv - Dohvaćanje pizze prema nazivu (npr. GET /pizze/Margherita)

router.get('/:naziv', (req, res) => {
    const naziv = req.params.naziv;
    const pizza = pizze.find(p => p.naziv.toLowerCase() === naziv.toLowerCase());

    if (!pizza) {
        return res.status(404).json({ message: `Pizza s nazivom '${naziv}' nije pronađena.`
});
    }

    res.status(200).json(pizza);
});

export default router;
```

Uključite `pizze` router u glavnu `index.js` datoteku:

```
// app/pizza-express/index.js

import pizzeRouter from './routes/pizze.js';
app.use('/pizze', pizzeRouter); // dodavanje prefiksa "/pizze" za svaki endpoint iz pizze.js
Routera
```

Otvorite Postman, dodajte odgovarajuće testove i provjerite ispravnost implementacije.



> Slika 1: Testiranje GET /pizze *endpointa* u Postmanu (odabrani prikaz `Preview` HTTP odgovora)

1.1.2 Implementacija `/narudzbe` `Router`

U ovaj `Router` dodajemo rutu za izradu nove narudžbe pizza.

Očekujemo sljedeći JSON format podataka u tijelu POST zahtjeva:

```
{
    "narucene_pizze": [
        {
            "naziv": "Margherita",
            "velicina": "srednja",
            "kolicina": 2
        },
        {
            "naziv": "Fantasia",
            "velicina": "jumbo",
            "kolicina": 1
        }
    ],
    "podaci_dostava": {
        "prezime": "Perić",
        "adresa": "Zagrebačka 15, Pula",
        "telefon": "091234567"
    }
}
```

Ukupnu cijenu narudžbe ćemo izračunati na poslužitelju na temelju naručenih pizza.

```
// app/pizza-express/routes/narudzbe.js

import express from 'express';
import { narudzbe, pizze } from '../data/data.js'; // učitavanje dummy podataka
const router = express.Router();

// POST /narudzbe - Izrada nove narudžbe pizza
router.post('/', (req, res) => {
    const { narucene_pizze, podaci_dostava } = req.body;
    if (!narucene_pizze || narucene_pizze.length === 0) {
        return res.status(400).json({ message: 'Nisu specificirane naručene pizze.' });
    }
    // Izračun ukupne cijene narudžbe
    let ukupna_cijena = 0;
    for (const narucena of narucene_pizze) {
        const pizza = pizze.find(p => p.naziv.toLowerCase() ===
narucena.naziv.toLowerCase());
        if (!pizza) {
            return res.status(400).json({ message: `Pizza s nazivom '${narucena.naziv}' nije
dostupna.` });
        }
        const cijena = pizza.cijene[narucena.velicina.toLowerCase()];
        if (!cijena) {
            return res.status(400).json({ message: `Veličina '${narucena.velicina}' nije
dostupna za pizzu '${narucena.naziv}'.` });
        }
        ukupna_cijena += cijena * narucena.kolicina;
    }
    ukupna_cijena = Number(ukupna_cijena.toFixed(2)); // zaokruživanje ukupne cijene na 2
decimale
});
```

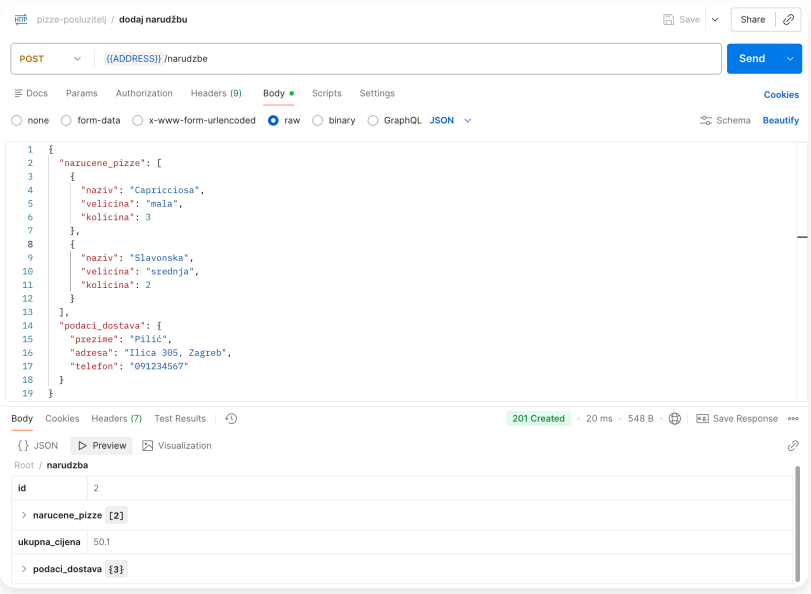
Nakon što smo validirali podatke iz HTTP zahtjeva i izračunali ukupnu cijenu narudžbe, stvorit ćemo novi zapis narudžbe i dodati ga u našu *in-memory* listu narudžbi.

```
ukupna_cijena = Number(ukupna_cijena.toFixed(2));

const nova_narudzba = {
    id: narudzbe.length + 1,
    narucene_pizze,
    ukupna_cijena,
    podaci_dostava
  };
  narudzbe.push(nova_narudzba);
  res.status(201).json({ message: 'Narudžba je uspješno kreirana.', narudzba: nova_narudzba
});
});

export default router;
```

Uključite `narudzbe` router u glavnu `index.js` datoteku:

```
// app/pizza-express/index.js

import narudzbeRouter from './routes/narudzbe.js';
app.use('/narudzbe', narudzbeRouter); // dodavanje prefiksa "/narudzbe" za svaki endpoint iz
narudzbe.js Routera
```

Otvorite Postman, dodajte odgovarajući test i provjerite ispravnost implementacije.



> Slika 2: Testiranje POST /pizze *endpointa* u Postmanu (odabrani prikaz `Preview` HTTP odgovora i JSON tijela zahtjeva)

To je to za sada! Prebacujemo se na izradu klijentske strane naše web aplikacije koristeći Vue.js.

2. Implementacija Vue.js klijentske strane

Izradit ćemo jednostavno Vue.js korisničko sučelje koristeći Tailwind CSS 4.1 CSS *framework* i Vite Build Tool.

TailwindCSS je vrlo popularan *utility-first CSS framework* (znači: koristi gotove CSS klase za stilizaciju elemenata umjesto pisanja prilagođenog CSS-a) koji omogućuje izradu korisničkih sučelja brzim sastavljanjem unaprijed definiranih CSS klasa niske razine (npr. `flex`, `pt-4`, `text-center`, `bg-blue-500`, itd.). Umjesto pisanja vlastitih CSS pravila, izgled i stil elemenata definiramo kombiniranjem gotovih klasa direktno u HTML/Vue datotekama.

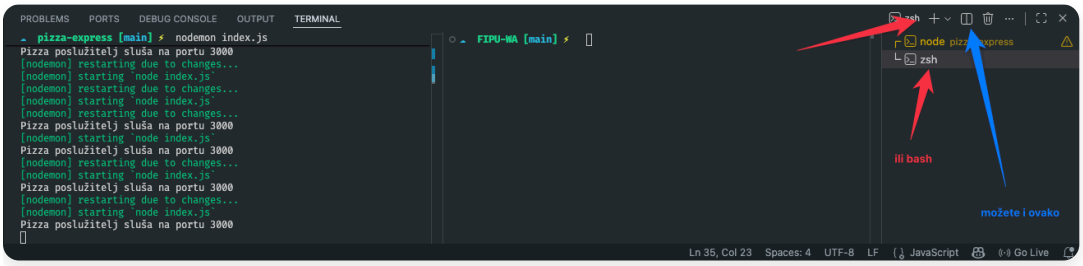
U osnovi, TailwindCSS djeluje kao alat koja nam pruža **gradivne blokove** kojima slažemo dizajn našeg korisničkog sučelja.

Vite je moderno razvojno okruženje i alat za izgradnju web aplikacija, optimiziran za rad s JavaScript okvirima kao što su Vue.js, React.js i Svelte. Temelji se na izuzetno brzom *dev server* mehanizmu koji omogućuje trenutačno osvježavanje tijekom razvoja, učinkovitu optimizaciju te brzu izgradnju aplikacija. Uz to, nudi izvrsnu podršku za TypeScript i JSX/TSX.

Iako je **Webpack** dugo bio standard u industriji, Vite je danas sve češći izbor zahvaljujući znatno boljim performansama i minimalnoj konfiguraciji potrebnoj za pokretanje projekta.

2.1 Konfiguracija Vue.js projekta s TailwindCSS-om i Vite-om

Pustite otvoren Express.js poslužitelj te u novom terminalu inicijalizirajte Vue.js projekt.



Prebacit ćemo se u `pizza-vue` direktorij i inicijalizirati Vue.js projekt koristeći Vite.

Otvorit će se interaktivni vodič za konfiguraciju projekta.

Možete odabrati što god želite, no preporuka je za početak uključiti samo `Router` i `Prettier`, te započeti s praznim projektom.

> Prettier je korisna VS Code ekstenzija za automatsko formatiranje koda prema definiranim pravilima, što pomaže u održavanju konzistentnog stila kodiranja. Preporuka je koristiti Prettier u svim projektima kako bi se izbjegle nesuglasice u stilu kodiranja među različitim članovima tima.

```
→ cd /app/pizza-vue
→ npm create vue@latest
```

```
Need to install the following packages:
create-vue@3.18.3
Ok to proceed? (y) y


> npx
> "create-vue"

┌  Vue.js - The Progressive JavaScript Framework
│
◇  Project name (target directory):
│  vue-project
│
◆  Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle
all, enter to confirm)
│  □ TypeScript
│  □ JSX Support
│  ■ Router (SPA development)
│  □ Pinia (state management)
│  □ Vitest (unit testing)
│  □ End-to-End Testing
│  □ ESLint (error prevention)
│  ■ Prettier (code formatting)

◇  Select experimental features to include in your project: (↑/↓ to navigate, space to
select, a to toggle all, enter to confirm)
│  none
│
◇  Skip all example code and start with a blank Vue project?
│  Yes
```

Nakon što je projekt inicijaliziran, primijetit ćete da ste dobili direktorij unutar `pizza-vue` direktorija s imenom koje ste odabrali (u našem slučaju `vue-project`).

Ovo možemo riješiti bash naredbom `mv` za premještanje svih datoteka iz poddirektorija u glavni `pizza-vue` direktorij:

```
→ pwd (#provjerite da ste u /app/pizza-vue)
→ mv vue-project/* vue-project/.* . (# premještanje svih neskrivenih (*) i skrivenih datoteka
(.*) iz vue-project u trenutni direktorij .)
```

> Hint: Kod instalacijskog Vue procesa, moguće je kao naziv projekta unijeti točku `.` kako bi se izbjeglo stvaranje dodatnog poddirektorija ili jednostavno pozvati instalacijski alat iz direktorija `app`.

Prije nastavka provjerite jesu lise sve datoteke prebacile. Ako jesu, `vue-project` direktorij sada možete obrisati.

Otvorite `package.json` datoteku i provjerite jesu li sve ovisnosti ispravno instalirane:

Primjer: `package.json` datoteke Vue.js projekta:

```
{
    "name": "vue-project",
    "version": "0.0.0",
    "private": true,
    "type": "module",
    "engines": {
        "node": "^20.19.0 || >=22.12.0"
    },
    "scripts": {
        "dev": "vite",
        "build": "vite build",
        "preview": "vite preview",
        "format": "prettier --write --experimental-cli src/"
    },
    "dependencies": {
        "vue": "^3.5.25",
        "vue-router": "^4.6.3"
    },
    "devDependencies": {
        "@vitejs/plugin-vue": "^6.0.2",
        "prettier": "3.6.2",
        "vite": "^7.2.4",
        "vite-plugin-vue-devtools": "^8.0.5"
    }
}
```

- Uočite `scripts` svojstvo u kojem su definirane različite CLI naredbe za razvoj i izgradnju Vue.js aplikacije.

Prije nego možemo pokrenuti razvojni poslužitelj, potrebno je instalirati sve ovisnosti navedene u `package.json` datoteci:

```
→ npm install
```

Primjer uspješne instalacije potrebnih paketa:

```
added 125 packages, and audited 126 packages in 24s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

`npm` naredbe pokrećemo iz terminala unutar `pizza-vue` direktorija, prefiksom `npm run <script-name>`:

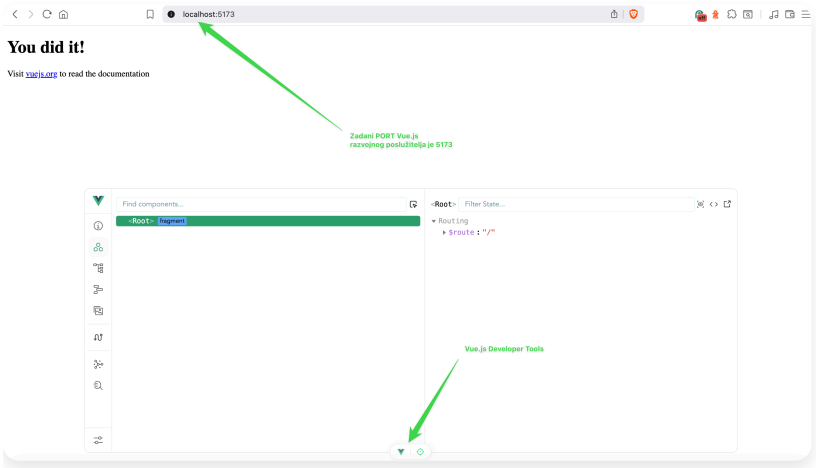
Primjer definiranih npm naredbi:

```
→ npm run dev (# pokretanje CLI naredbe vite)
→ npm run build (# pokretanje CLI naredbe vite build)
→ npm run format (# pokretanje CLI naredbe prettier --write --experimental-cli src/)
```

Sada možemo pokrenuti razvojni poslužitelj koristeći naredbu `npm run dev`:

```
→ npm run dev

# ili samo

→ vite
```

Otvorite web preglednik i posjetite `http://localhost:5173/` kako biste vidjeli početnu stranicu Vue.js aplikacije.



> Slika 3: Početna stranica Vue.js aplikacije pokrenute s Vite razvojnim poslužiteljem na zadanom portu `5173`. Vue.js developer tools možete otvoriti putem preglednika za lakše debugiranje Vue komponenti kraticom `SHIFT + ALT/OPT + D`.

> Hint: Isto možete provjeriti i u Postmanu. Možete zamisliti Vite kao specijalizirani poslužitelj koji isporučuje HTML stranicu s ugrađenim JavaScript modulima koji onda iscrtavaju web stranicu u pregledniku. Ipak, osnovni sadržaj stranice bez dinamičkih modula možete vidjeti i u Postmanu.

2.2 TailwindCSS konfiguracija

Sada ćemo **konfigurirati TailwindCSS** unutar našeg Vue.js projekta.

Zaustavite Vite poslužitelj slanjem `kill` signala (`CTRL + C`) u aktivnom terminalu.

Pokrenite sljedeće naredbe za instalaciju TailwindCSS-a i njegovih ovisnosti:

> Napomena: Za vrijeme pisanja ove skripte, najnovija verzija TailwindCSS-a je 4.1, a za Vite postoji i službeni plugin `@tailwindcss/vite` koji olakšava integraciju. Uvijek je dobro pratiti službene dokumentacije kako biste bili sigurni da se koraci instalacije nisu promijenili.

```
→ pwd (#provjerite da ste u /app/pizza-vue)
→ npm install tailwindcss @tailwindcss/vite
```

Otvorite `vite.config.js` datoteku i dodajte `@tailwindcss/vite` plugin kako bi TailwindCSS mogao biti pravilno integriran s Viteom:

```
// vite.config.js
import { defineConfig } from 'vite';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
    plugins: [
        tailwindcss() // dodajte ovaj redak
    ]
});
```

Nakon toga, dodat ćemo osnovnu konfiguracijsku datoteku `assets/tailwind.css` za TailwindCSS unutar našeg `src` direktorija:

```
→ cd src
→ mkdir assets
→ touch assets/tailwind.css
```

Unutar datoteke `assets/tailwind.css`, dodajte samo jedan redak za uvoz osnovnih TailwindCSS stilova:

```
@import 'tailwindcss';
```

Sada ćemo osigurati da se `tailwind.css` datoteka učitava unutar glavne `main.js` datoteke Vue.js aplikacije:

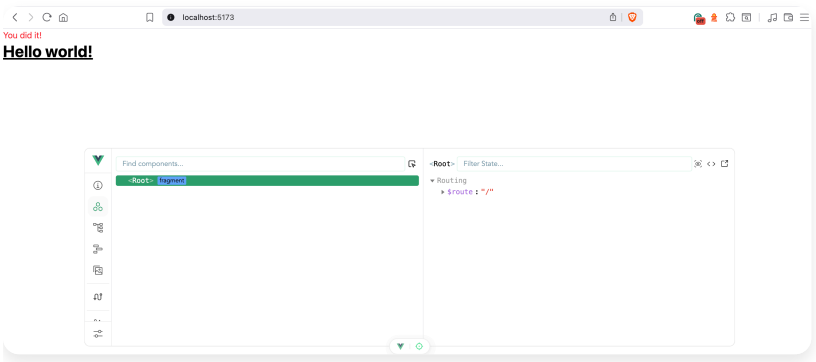
```
// app/pizza-vue/src/main.js

import './assets/tailwind.css';
```

To je to! Sada možete ponovno pokrenuti Vite razvojni poslužitelj i provjeriti je li TailwindCSS ispravno integriran na način da dodate neke osnovne TailwindCSS klase u `App.vue` datoteku:

```
<!-- app/pizza-vue/src/App.vue -->
<template>
    <h1 class="text-red-500">You did it!</h1>
    <h1 class="text-3xl font-bold underline">Hello world!</h1>
</template>
```

> Podsjetnik za TailwindCSS: Programsko inženjerstvo - Vue Osnove skripta, autor: Alesandro Žužić, mag. inf.

Pokrenite Vite razvojni poslužitelj, trebali biste vidjeti crveni tekst You did it! i veliki podcrtani naslov **Hello world!**.



> Slika 4: Ispravno integriran TailwindCSS u Vue.js aplikaciju (prikaz u pregledniku)

Općenito, klase za Tailwind CSS ne želite mijenjati direktno u CSS kodu, već koristiti i kombinirati predefinirane klase koje dolaze s Tailwindom. Na primjer, klasa `text-red-500` postavlja boju teksta na crvenu, dok `text-3xl`, `font-bold` i `underline` definiraju veličinu fonta, debljinu i podcrtavanje teksta.

Preporuka je naučiti služiti se TailwindCSS dokumentacijom.

Uspješno smo konfigurirali Vue.js projekt s TailwindCSS-om i Viteom! 🚀 Naša web aplikacija sada se sastoji od dva dijela:

1. **Express.js poslužitelj** koji upravlja podacima o pizzama i narudžbama
2. **Vue.js klijentska aplikacija** koja će komunicirati s Express poslužiteljem i pružiti korisničko sučelje za naručivanje pizza

Provjerite imate li ispravnu datotečnu strukturu web aplikacije:

```
app
├── pizza-express
│   ├── node_modules
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   ├── data
│   │   └── data.js
│   ├── routes
│   │   ├── pizze.js
│   │   └── narudzbe.js
│   └── .gitignore
└── pizza-vue
    ├── node_modules
    ├── public
    ├── .vscode (opcionalno)
    ├── src
    │   ├── assets
    │   │   └── tailwind.css
    │   ├── router
    │   │   └── index.js
    │   ├── App.vue
    │   └── main.js
    ├── vite.config.js
    ├── package.json
    ├── package-lock.json
    ├── .prettierrc.json (opcionalno)
    ├── index.html
    ├── jsconfig.json
    ├── README.md
    ├── .gitattributes (opcionalno)
    └── .gitignore
```

> Hint: Kada radite na većim projektima u VS Code-u, može postati nezgodno pratiti i raditi na većem broju datoteka i direktorija. Preporuka je instalirati neku ekstenziju koja grafički uređuje datotečnu strukturu (`Explorer tab`). Preporuka: Material Icon Theme.

> Hint 2: Dvije korisne kratice u VS Code-u kada radite na većim projektima su:

- `CTRL/CMD + P` - Otvara brzi **pretraživač datoteka** unutar projekta (krenite unositi naziv datoteke bez obzira gdje se nalazi)
- `CTRL/CMD + Shift + F` - Otvara **globalni pretraživač teksta** unutar svih datoteka projekta (krenite unositi tekst/kod koji tražite bez obzira gdje se nalazi)

Prije nego nastavite, preporučuje se da spremite *commitate* sve promjene u vaš Git repozitorij kako biste imali sigurnosnu kopiju trenutnog stanja projekta.

> Napomena: Kod izrade projekta iz kolegija, *frontend* i *backend* dijelove aplikacije morat ćete **verzionirati u zasebnim repozitorijima**. Da ne kompliciramo, za potrebe ove vježbe, ostavit ćemo sve u jednom repozitoriju.

2.3 Dodavanje osnovnih komponenti korisničkog sučelja

Želimo izraditi grafičko korisničko sučelje gdje korisnik može pregledavati dostupne pizze i naručiti ih.

Na Express poslužitelju već imamo implementirane potrebne *endpointove* za dohvaćanje podataka o pizzama i slanje narudžbi.

Želimo korisniku prikazati grafičko sučelje gdje može vidjeti sve dostupne pizze i njihove detalje (naziv, sastojke, cijene) te omogućiti odabir veličine i količine za svaku pizzu koju želi naručiti.

U kontekstu poslužitelja, zamislite da moramo prvo implementirati GET `/pizze` endpoint koji vraća popis svih pizza u JSON formatu te lijepo prikazati te podatke u Vue.js aplikaciji. Nakon toga ćemo implementirati funkcionalnost naručivanja pizza putem POST `/narudzbe` *endpointa*.

PizzaList.vue komponenta

Izradit ćemo Vue komponentu `PizzaList.vue` koja će dohvaćati i prikazivati popis dostupnih pizza s Express poslužitelja.

Kako *frontend* dizajn korisničkog sučelja nije tema ovog kolegija, upotrijebit ćemo gotovi *tailwind-HTML* predložak te raditi na funkcionalnostima Vue komponente. Ako hoćete, možete uređivati stilove prema vlastitim željama i/ili izraditi vlastiti UI dizajn.

Sve predloške možete pronaći na GitHubu kolegija: WA3 - Razmjena podataka između klijenta i poslužitelja/vue-templates.

- odaberite `pizza-list.html` datoteku i kopirajte njen sadržaj

Stvorite `components` direktorij unutar `src` direktorija Vue.js projekta i dodajte `PizzaList.vue` datoteku:

```
→ cd src
→ mkdir components
→ touch components/PizzaList.vue
```

Struktura `components` direktorija:

```
app
└── pizza-vue
    ├── src
    │   ├── assets
    │   │   └── tailwind.css
    │   ├── components
    │   │   └── PizzaList.vue
```

Vue3 komponenta sastoji se od barem dva obavezna dijela:

1. **Template (HTML) dio** - definira HTML strukturu komponente (unutar `<template>` taga) - gotovo uvijek koristimo
2. **Script (JS) dio** - definira logiku komponente (unutar `<script setup>` taga) - gotovo uvijek koristimo

Ipak, moguće je dodati i CSS stilove unutar Vue komponente:

3. **Style (CSS) dio** - definira stilove komponente (unutar `<style>` taga) - rjeđe koristimo (npr. ako koristimo TailwindCSS, rijetko kad želimo pisati prilagođeni (custom) CSS)

Sintaksa Vue3 komponente:

```
<template>
    <!-- HTML struktura komponente -->
</template>

<script setup>
    // Logika komponente (JS/TS)
</script>

<style>
    /* Stilovi komponente (CSS) */
</style>
```

Samo ćemo kopirati HTML predložak iz `pizza-list.html` datoteke u **template** dio `PizzaList.vue` komponente i ukloniti `body` tag budući da HTML kod Vue komponente mora biti unutar `<template>` taga.

Vidjet ćete mnogo `class` atributa u kojima su pohranjene TailwindCSS klase za stilizaciju elemenata.

Komponenta sadrži strukturu i stil koji iscrtavaju **karticu pizze**, a ponavljaju se 3 puta.

Isječak kartice za pizzu iz `pizza-list.html`:

```
<div class="bg-inherit rounded-xl overflow-hidden">
    <div class="w-full h-48 flex items-center justify-center bg-white">
        <!-- Slika s interneta -->
        <img src="https://www.freeiconspng.com/uploads/pizza-png-1.png" alt="Pizza Image 1"
class="w-full h-full object-contain" />
    </div>

    <div class="p-6">
        <div class="flex items-center space-x-3 mb-4">
            <!-- Naziv -->
            <h2 class="text-lg font-bold text-orange-500 tracking-wide">Pizza 1</h2>
            <!-- Sastojci -->
            <div class="flex space-x-2">
                <div class="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-
center text-slate-50 font-semibold text-xs">Icon</div>
                <div class="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-
center text-slate-50 font-semibold text-xs">Icon</div>
                <div class="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-
center text-slate-50 font-semibold text-xs">Icon</div>
            </div>
        </div>
        <!-- Cijene za svaku veličinu -->
        <div class="space-y-2">
            <div class="flex justify-between text-gray-700">
                <span class="font-medium">Mala</span>
                <span>€00.00</span>
            </div>

            <div class="flex justify-between text-gray-700">
                <span class="font-medium">Srednja</span>
                <span>€00.00</span>
            </div>

            <div class="flex justify-between text-gray-700">
                <span class="font-medium">Jumbo</span>
                <span>€00.00</span>
            </div>
        </div>
    </div>
</div>
```

> Hint: Pokušajte uočiti dijelove HTML koda gdje su prikazani naziv pizze, slika, sastojci i cijene za različite veličine. Važno je razumjeti strukturu HTML-a čak i ako ne razumijete sve detalje dizajna (u ovom slučaju TailwindCSS klasa), budući da ćemo kasnije trebati dinamički mijenjati te dijelove koda koristeći Vue.js na temelju podataka dohvaćenih s poslužitelja.

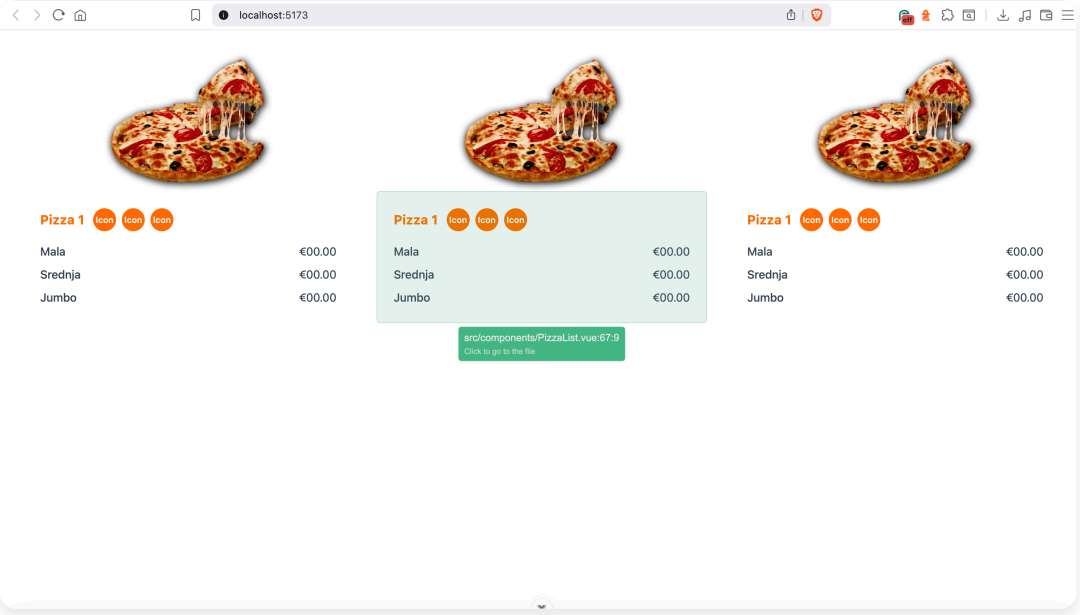
`PizzaList.vue` komponenta sada izgleda ovako:

```
// app/pizza-vue/src/components/PizzaList.vue

<template> ...kopirani HTML predložak iz pizza-list.html datoteke... </template>

<script setup></script>

<style></style>
```

Otvorite `App.vue` datoteku i uvezite `PizzaList.vue` komponentu unutar `script setup` dijela, a nakon toga je pozovite unutar `template` dijela.

Na ovaj način ćemo iscrtati `PizzaList` komponentu unutar glavne aplikacijske komponente `App.vue`.

```
// app/pizza-vue/src/App.vue

<template>
    <PizzaList />
</template>
```



> Slika 5: Prikaz u web pregledniku: HTML predložak iz `PizzaList.vue` komponente iscrtava se u pregledniku koristeći TailwindCSS stilove.

> Hint: Možete otvoriti *Vue.js developer tools* u pregledniku kako biste lakše pratili strukturu Vue komponenti ili HTML elemente u DOM stablu. Pokušajte izmijeniti neke TailwindCSS klase unutar `PizzaList.vue` komponente i proučite kako se mijenja izgled stranice u web pregledniku.

Dodat ćemo još pozadinsku sliku i zaglavlje (*header*) naše aplikacije.

Pozadinsku sliku `background.png` možete također pronaći u `WA3 - Razmjena podataka između klijenta i poslužitelja/vue-templates/` direktoriju.

Slike koje želimo koristiti u Vue.js aplikaciji (čitaj: koje želimo da Vite razvojni poslužitelj servira) moraju biti smještene unutar `public` direktorija Vue.js projekta.

Struktura `public` direktorija:

```
app
└── pizza-vue
    ├── public
    │   └── background.png
```

> Hint: Statične datoteke koje stavljamo unutar `public` direktorija postaju javni resursi dostupni svima koji pristupe našoj web aplikaciji. Vite će automatski poslužiti te datoteke kada se aplikacija pokrene u razvojnom ili produkcijskom okruženju, npr. `http://localhost:5173/background.png`.

Pozadinsku sliku možemo postaviti na više načina, a ako želimo ostati vjerni TailwindCSS-u, možemo dodati nekoliko gotovih klasa na prvi `div` unutar `PizzaList.vue` komponente:

Pronađite prvi `div`:

```
<div class="mx-auto bg-linear-to-br min-h-screen p-8"></div>
```

Dodajemo sljedeće TailwindCSS klase u ovaj `div`:

```
bg-[url('/background.png')] bg-cover bg-center bg-no-repeat
```

Rekli smo da Vue.js može pročitati slike iz `public` direktorija koristeći relativnu putanju počevši od korijenskog modula web aplikacije (što je u ovom slučaju `/`). Iz tog razloga ne moramo navoditi cijelu putanju do slike, već samo `/background.png`.

Header.vue komponenta

Idemo na jednak način dodati i zaglavlje (*header*) naše aplikacije.

Stvorite `Header.vue` datoteku unutar `components` direktorija:

```
→ cd src/components
→ touch Header.vue
```

Struktura `components` direktorija sada izgleda ovako:

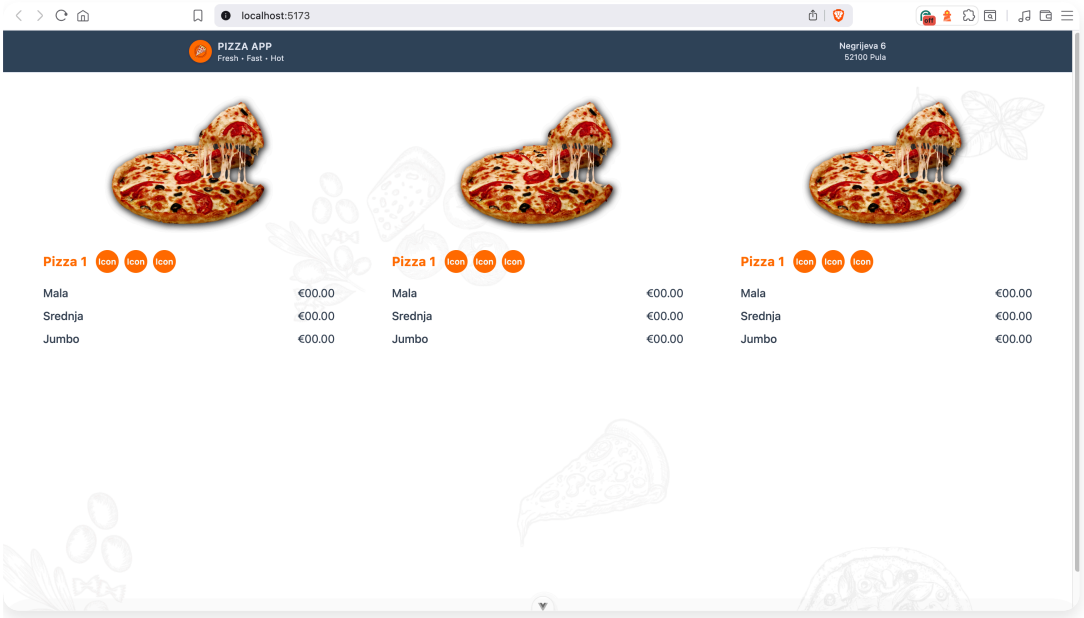
```
app
└── pizza-vue
    ├── src
    │   ├── assets
    │   │   └── tailwind.css
    │   ├── components
    │   │   ├── Header.vue
    │   │   └── PizzaList.vue
```

Kopirajte HTML predložak iz `header.html` datoteke (nalazi se u `WA3 - Razmjena podataka između klijenta i poslužitelja/vue-templates/`) u `Header.vue` komponentu.

Uvezite komponentu unutar `App.vue` datoteke i unesite je iznad `PizzaList` komponente kako bi se zaglavlje prikazalo na vrhu stranice (iznad popisa pizza).

```
// app/pizza-vue/src/App.vue

<template>
    <header />
    <PizzaList />
</template>
<script setup>
    import Header from './components/Header.vue';
    import PizzaList from './components/PizzaList.vue';
</script>
```

To je to! Vaša web aplikacija sada bi trebala imati pozadinsku sliku iza sadržaja i zaglavlje (*header*) na samom vrhu.



> Slika 6: Prikaz u web pregledniku: `PizzaList.vue` komponenta s pozadinskom slikom i `Header.vue` komponentom

Implementacija odabira pizze

Reaktivnost (*eng. reactivity*) predstavlja jedan od temeljnih koncepata u Vue.js ekosustavu. Ona omogućuje da se korisničko sučelje automatski osvježi svaki put kada dođe do promjene podataka u pozadini, čineći time reaktivnost ključnim mehanizmom na kojem počiva cijeli Vue.js okvir.

Ukratko, koristimo **dvije ključne riječi** za definiranje reaktivnih podataka u Vue3 komponentama: `ref` i `reactive`.

Ove funkcije svojevrsni su omotači (*wrapperi*) koji omogućuju Vue.js da "prati promjene vrijednosti varijabli i objekata" te automatski ažurira DOM (*Document Object Model*) kada dođe do promjene u podacima.

Sintaksa:

```
const dinamicka_varijabla = ref(pocetna_vrijednost); // najčešće za primitivne tipove
podataka, ali može se koristiti i s referentnim tipovima

// koristi se samo s referentnim tipovima podataka (objekti)
const dinamicki_objekt = reactive({
    // svojstvo1: vrijednost1,
    // svojstvo2: vrijednost2
}); // za složene tipove podataka (objekte)
```

Primjer definiranja reaktivne varijable s `ref`:

- trenutnoj vrijednosti `ref` objekta pristupamo preko `.value` svojstva

```
import { ref, reactive } from 'vue';

let count = ref(0); // može se koristiti s bilo kojim tipom podataka

console.log(count); // Ispisuje ref objekt
console.log(count.value); // Ispisuje trenutnu vrijednost (0)

//count++ // NE ! neispravno povećanje vrijednosti za 1
console.log(count.value); // undefined

count.value++; // DA: ispravno povećanje vrijednosti za 1
console.log(count.value); // 1
```

Primjer definiranja reaktivne varijable s `reactive`:

- u usporedbi s `ref`, `reactive` će učiniti cijeli objekt dubinski reaktivnim, što znači da možemo mijenjati i njegova svojstva izravno - bez potrebe za `.value` svojstvom.

```
const user = reactive({
    // složeni tipovi podataka (objekti)
    name: 'Marko',
    prezime: 'Marković',
    dob: 30
});

// uočite da ne koristimo .value
console.log(user.name); // Ispisuje 'Marko'
user.dob++; // Ispravno mijenja dob na 31
console.log(user.dob); // Ispisuje 31

const settings = reactive({
    // složeni tipovi podataka (objekti)
    tema: 'svijetla',
    notifikacije: true,
    jezici: ['hrvatski', 'engleski']
});
```

> Više o reaktivnosti u Vue.js možete pročitati u službenoj dokumentaciji: Reactivity Fundamentals.

Otvorite `PizzaList.vue` komponentu i unutar `<script setup>` dijela uvezite `ref` funkciju iz `vue` paketa:

- Definirat ćemo reaktivnu varijablu `odabrana_pizza` i postavit joj vrijednost na `null`.
- Definirat ćemo i jednostavnu funkciju `odaberiPizzu(pizza_naziv)` koja će promijeniti vrijednosti `odabrana_pizza` varijable na proslijeđeni `pizza_naziv` string.

Primjer:

```
// app/pizza-vue/src/components/PizzaList.vue
import { ref } from 'vue';
const odabrana_pizza = ref(null); // reaktivna varijabla za pohranu naziva odabrane pizze

function odaberiPizzu(pizza_naziv) {
    odabrana_pizza.value = pizza_naziv; // postavljanje naziva odabrane pizze
    console.log('Odabrana pizza:', odabrana_pizza.value); // ispis u konzolu (provjerite)
}
```

Unutar `template` dijela `PizzaList` komponente, pronađite pizze i izmijenite im nazive na proizvoljne vrijednosti.

Funkcije možemo pozivati iz HTML dijela komponente koristeći `@click` Vue direktivu na gumbu za svaku pizzu. Ovismo gdje stavimo direktivu, ona će "obuhvatiti" taj HTML element i pozvati funkciju klikom na taj element.

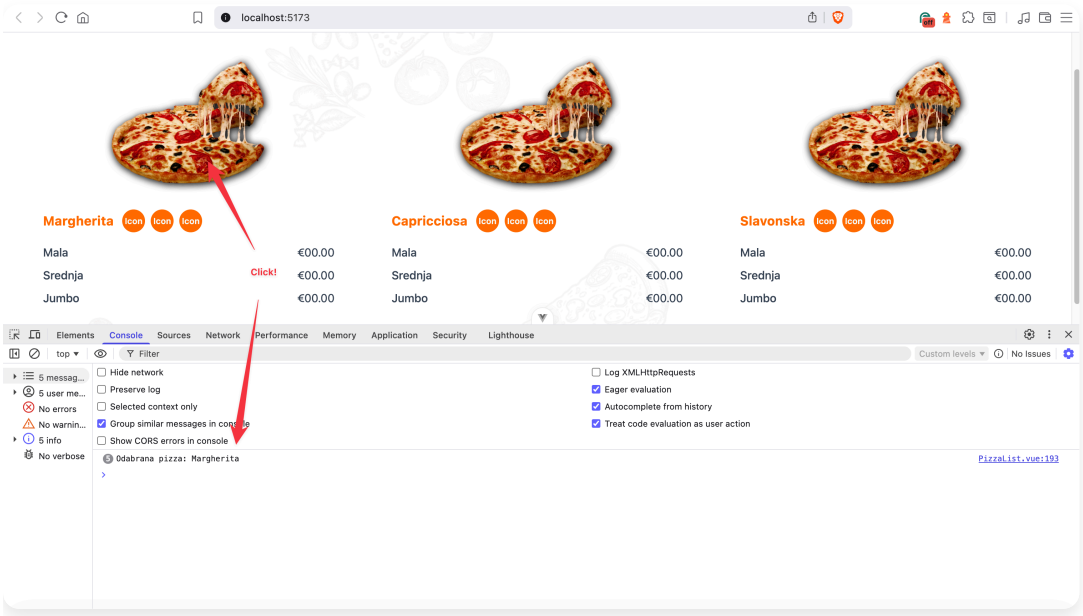
Sintaksa `@click` direktive:

```
<!-- pozivanje funkcije bez argumenata -->
<button @click="naziv_funkcije()">Klikni me</button>
<!-- pozivanje funkcije s argumentima -->
<button @click="naziv_funkcije(arg1, arg2)">Klikni me</button>

<!-- primjer sa string argumentom -->
<button @click="odaberiPizzu('Margherita')">Odaberi Margherita pizzu</button>
```

> Hint: Ako u argumentima koristite stringove, preporučuje se da unutar dvostrukih navodnika (`""`) u `@click` direktivi upotrebljavate jednostruke navodnike (`''`). Time ćete izbjeći probleme s parsiranjem HTML-a i osigurati pravilno izvršavanje direktive.

Primjer: Izmijenjeni naziv pizze i dodani `@click` event:

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->

<!-- direktiva @click dodana u glavni div -->
<div @click="odaberiPizzu('Margherita')">
    <div class="bg-inherit rounded-xl overflow-hidden">
        <div class="w-full h-48 flex items-center justify-center bg-inherit">
            <img src="https://www.freeiconspng.com/uploads/pizza-png-1.png" alt="Pizza Image"
class="w-full h-full object-contain" />
        </div>
    </div>
    ... ostatak HTML-a pizze
</div>
```



> Slika 7: Dodan `@click` event na HTML element pizze u `PizzaList.vue` komponenti (prikaz u pregledniku)

Kako bismo naznačili korisniku da je kliknuo na određenu pizzu, možemo promijeniti stil kartice odabrane pizze koristeći **uvjetno dodavanje/oduzimanje TailwindCSS klasa**.

Uvjetnu izmjenu klase u Vue.js komponenti možemo napraviti koristeći `:class` direktivu.

> Hint: Općenito, dodavanjem dvotočke (`:`) ispred atributa HTML elementa, Vue.js zna da treba interpretirati vrijednost tog atributa kao JavaScript izraz. Ovo je moguće na većini HTML atributa, uključujući `class`, `style`, `src`, `href`, itd.

Sintaksa:

```
<div :class="{'niz-klasa-ako-je-uvjet-true': uvjet, 'drugi-niz-klasa-ako-je-drugi_uvjet-true': drugi_uvjet}">
    <!-- sadržaj -->
</div>

<!--ili-->

<div :class="['zadani-niz-klasa', uvjet ? 'dodatni-niz-klasa-ako-je-uvjet-true' : 'alternativni-niz-ako-je-uvjet-false']">
    <!-- sadržaj -->
</div>
```

Primjeri:

```
<!-- Prva sintaksa -->
<div
    :class="{ 'bg-blue-500': isActive,
              'bg-gray-500': !isActive }"
></div>

<!-- Alternativna sintaksa -->
<div
    :class="[
          'p-4 rounded',
          isActive ? 'bg-blue-500 text-white' : 'bg-gray-200 text-black',
        ]"
></div>
```

Uvjetni izraz u našem slučaju može biti: `odabrana_pizza == 'neki_naziv_pizze'`

- ako je `odabrana_pizza` == `null` ili `""`, nijedna pizza nije odabrana i ne iscrtavamo dodatne TailwindCSS klase
- ako je `odabrana_pizza` == nekom nazivu pizze, dobivamo `dodatni-niz-klasa-ako-je-uvjet-true`

Primjer: Niz TailwindCSS klasa koje ćemo dodati ako je pizza odabrana:

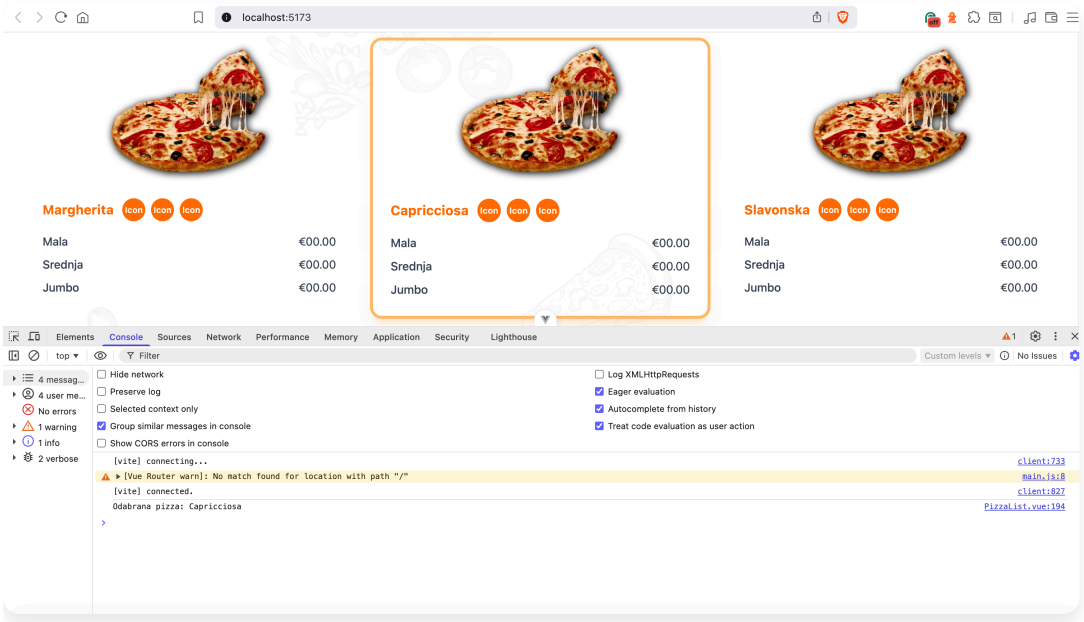
```
ring-4 ring-orange-300 shadow-lg shadow-orange-300/50 scale-[1.02]
```

Promijenit ćemo i zadanu klasu kako bismo dobili **tranzicijski efekt** prilikom odabira pizze:

```
<div
    :class="[
          'bg-inherit rounded-xl overflow-hidden cursor-pointer transition-all duration-300',
          odabrana_pizza === 'Slavonska'
            ? 'ring-4 ring-orange-300 shadow-lg shadow-orange-300/50 scale-[1.02]'
            : 'hover:scale-[1.01]',
        ]"
    ...
></div>
```

- **Zadana klasa** je prva: `bg-inherit rounded-xl overflow-hidden cursor-pointer transition-all duration-300`
- **Ako je pizza odabrana**, dodaju se klase iza ternarnog operatora (`?`): `ring-4 ring-orange-300 shadow-lg shadow-orange-300/50 scale-[1.02]`
- **Ako pizza nije odabrana**, dodaje se samo `hover:scale-[1.01]`

Možemo prebaciti `@click` direktivu na cijeli ovaj `div` budući da želimo da se pizza odabere kada korisnik klikne bilo gdje unutar kartice pizze.



> Slika 8: Uvjetno dodavanje TailwindCSS klasa za označavanje odabrane pizze u `PizzaList.vue` komponenti (prikaz u pregledniku)

Za sada toliko od dizajna. Idemo napokon implementirati komunikaciju s Express poslužiteljem kako bismo dohvatili stvarne podatke o pizzama i prikazali ih dinamički unutar `PizzaList.vue` komponente.

3. Axios i komunikacija s Express poslužiteljem

Za komunikaciju s Express poslužiteljem imamo na raspolaganju više opcija. Moguće je koristiti i `fetch` API koji smo upoznali na *Skriptnim jezicima* i *Programskom inženjerstvu*, međutim kroz neke vanjske biblioteke možemo definirati koncizniju sintaksu za slanje HTTP zahtjeva te rukovanje odgovorima.

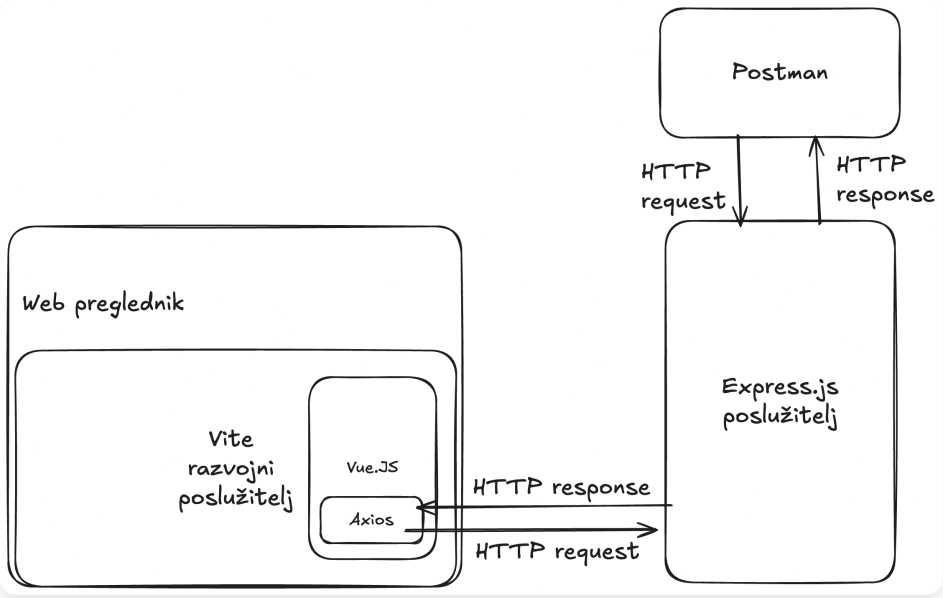
Jedna od takvih biblioteka je i Axios.

Axios je **HTTP klijent za Node i web preglednik** koji se bazira na sintaksi `Promise` objekata.

Instalirajte Axios unutar Vue.js projekta:

```
→ cd /app/pizza-vue
→ npm install axios
```

Prije nastavka, pogledajte ilustraciju koja prikazuje **razmjenu podataka između klijenta i poslužitelja** (odnosno između Vue.js aplikacije i Express poslužitelja), jer smo do sada uveli niz novih pojmova. Obratite pozornost gdje se u ovom procesu nalazi Axios biblioteka.



> Slika 9: Ilustracija razmjene podataka između klijenta (Vue.js aplikacije) i poslužitelja (Express.js aplikacije) koristeći Axios HTTP klijent

Prijetimo se ukratko sintakse `Promise` objekata kako bismo lakše razumjeli kako Axios funkcionira.

```
// Izrada novog Promise objekta
const myPromise = new Promise((resolve, reject) => {
    // asinkroni kod koji će na kraju pozvati resolve() ili reject() ovisno o ishodu
    let success = true; // primjer "super-jednostavne operacije koja se uvijek uspješno
izvrši"
    if (success) {
        resolve('Uspjeh!'); // poziva se ako je operacija uspješna
    } else {
        reject('Greška!'); // poziva se ako je došlo do greške
    }
});
```

Promise objekt predstavlja buduću vrijednost (*future*) koja može biti u jednom od tri stanja: **ispunjena** (`resolved`), **odbijena** (`rejected`) ili još uvijek **na čekanju/u tijeku** (`pending`).

> Drugim riječima, Promise nam omogućuje da radimo s asinkronim operacijama na način koji je sličniji sinkronom kodu, čineći ga lakšim za čitanje i održavanje. `pending` stanje ovdje konkretno znači da mrežna operacija (HTTP zahtjev) prema našem Express poslužitelju još nije izvršena, odnosno da poslužitelj još uvijek nije obradio zahtjev i vratio HTTP odgovor. `resolved` predstavlja uspješno izvršenu mrežnu operaciju (*non-500* odgovor poslužitelja), dok `rejected` označava da je došlo do greške tijekom mrežne operacije (npr. poslužitelj nije dostupan, došlo je do *timeouta*, itd.).

Eventualne rezultate Promise objekata obrađujemo metodama `.then()` i `.catch()`:

```
myPromise
    .then(result => {
        console.log(result); // ispisuje 'Uspjeh!' ako je resolve()
    })
    .catch(error => {
        console.error(error); // ispisuje 'Greška!' ako je reject()
    });
```

Također, možemo koristiti i `async/await` sintaksu za rad s Promise objektima:

- tada moramo koristiti `try/catch/finally` blok za "hvatanje" eventualnih grešaka

```
async function runAsyncTask() {
    try {
        const result = await myPromise; // čeka da se Promise riješi
        console.log(result); // ispisuje 'Uspjeh!' ako je Promise.resolve(), tj. ako se
Promise rezolvira
    } catch (error) {
        console.error(error); // ispisuje 'Greška!' ako je Promise.reject(), tj. ako se
Promise odbije
    }
}

runAsyncTask(); // pozivanje asinkrone funkcije runAsyncTask()
```

`axios` objekt je Promise koji ima **implementirane metode za slanje različitih HTTP zahtjeva**: `axios.get()`, `axios.post()`, `axios.put()`, `axios.delete()`, itd.

Tijelo `axios` Promise objekta postaje asinkroni HTTP zahtjev koji šaljemo na određeni URL (odgovarajući endpoint na `express-server` poslužitelju), a odgovor na taj zahtjev obrađujemo u `.then()` metodi ili koristeći `await` unutar asinkrone funkcije.

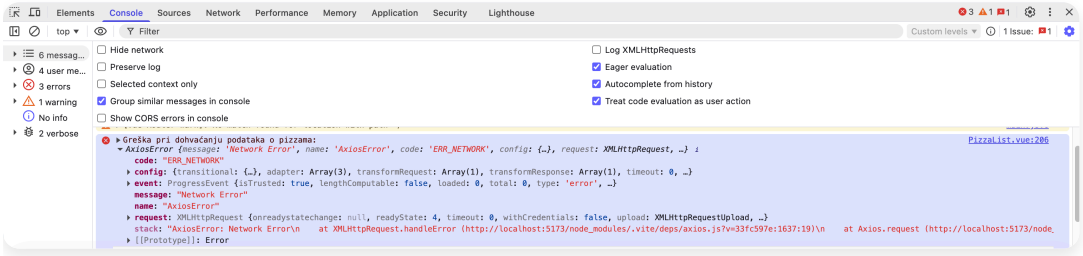
Dakle, kako bismo poslali HTTP GET zahtjev na `http://localhost:3000/pizze` endpoint i dohvatili popis svih pizza, možemo koristiti sljedeći kod:

```
// app/pizza-vue/src/components/PizzaList.vue

import axios from 'axios';

axios
    .get('http://localhost:3000/pizze') // slanje GET zahtjeva na /pizze endpoint
    // obrada uspješnog odgovora
    .then(response => {
        console.log(response.data); // ispisuje podatke o pizzama iz odgovora
    })
    // obrada greške
    .catch(error => {
        console.error('Greška pri dohvaćanju podataka o pizzama:', error);
    });
```

Ubacite ovaj kod u `PizzaList.vue` komponentu i pokušajte osvježite stranicu u pregledniku - na taj način ćete poslati GET zahtjev na Express poslužitelj i ispisati odgovor u konzolu web preglednika.

3.1 CORS politika

Nažalost, ako pokušate pokrenuti ovaj kod odmah, vjerojatno ćete dobiti grešku vezanu uz CORS (*Cross-Origin Resource Sharing*) politiku u pregledniku.

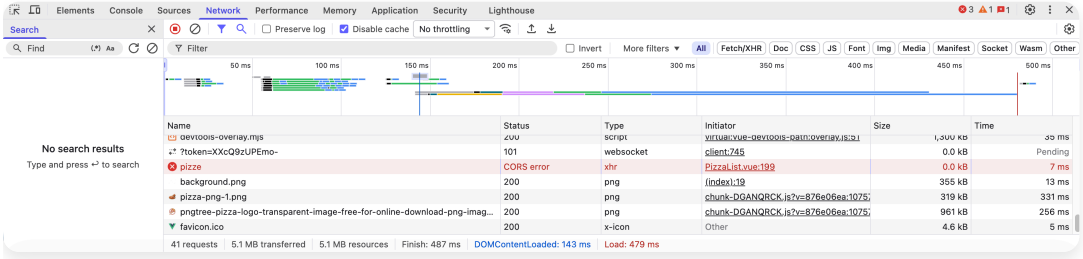


> Slika 10: CORS greška u konzoli preglednika prilikom pokušaja slanja HTTP zahtjeva s Vue.js aplikacije na Express poslužitelj

Možemo se dodatno uvjeriti da je došlo do greške tako da otvorimo *Network* tab u *developer toolsu* preglednika i pogledamo detalje HTTP zahtjeva.

Network tab pokazuje sve mrežne zahtjeve koje je web stranica napravila, uključujući HTTP zahtjeve **prema našem Express poslužitelju**, ali i **Vite razvojnom poslužitelju**.

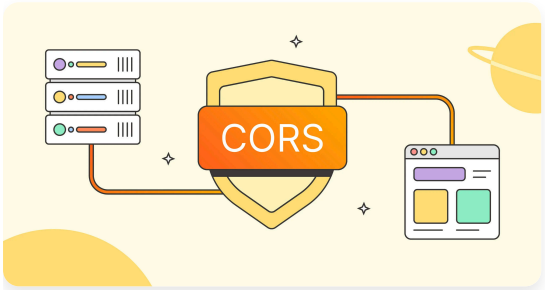
Osvježite ponovo stranicu i pronađite neuspjeli zahtjev prema Expressu obojen u crveno.



> Slika 11: Neuspjeli HTTP GET zahtjev s Vue.js aplikacije na Express poslužitelj zbog CORS politike (prikaz u Network tabu *developer toolsu* preglednika)

Ako otvorite detalje mrežnog zahtjeva, vidjet ćete detalje o HTTP zahtjevu i poslana zaglavlja (*request headers*). Međutim, nećete vidjeti podatke o HTTP odgovoru jer je preglednik blokirao pristup tim podacima zbog CORS politike. Ipak, statusni kod odgovora je `200 OK`, što znači da je poslužitelj ispravno obradio zahtjev.

CORS je **sigurnosna značajka web preglednika** koja kontrolira kako web stranice/aplikacije mogu komunicirati/zatražiti određene resurse preko poslužitelja koji se nalazi na drugoj domeni (ili samo portu u našem slučaju).



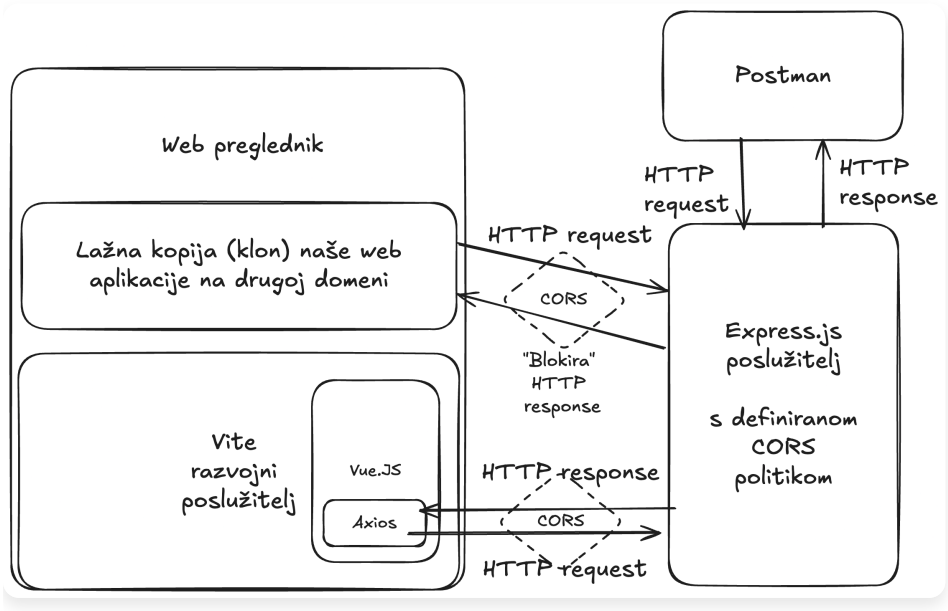
Drugim riječima, CORS politikom definira se smije li razvojni poslužitelj na domeni `http://localhost:5173` (Vite poslužitelj) komunicirati s Express poslužiteljem na domeni `http://localhost:3000`.

Ako smije, tada će web preglednik dopustiti da se HTTP zahtjev izvrši i da se podaci iz odgovora proslijede Vue.js aplikaciji koju izvršava razvojni poslužitelj. U suprotnom, preglednik će blokirati pristup podacima iz odgovora i prikazati CORS grešku u konzoli `(No 'Access-Control-Allow-Origin' header is present on the requested resource)`.

Ovo je **sigurnosni mehanizam** kojim možemo spriječiti zlonamjerne web aplikacije da pristupaju resursima na *backend* poslužiteljima bez dopuštenja i jako je dobra praksa implementirati ga na produkcijskim sustavima, uz adekvatnu dodatnu autorizaciju HTTP zahtjeva.

Konkretno, mi **moramo CORS politiku definirati na Express poslužitelju** kako bismo dopustili zahtjeve samo s naše Vue.js aplikacije.

Sljedeća ilustracija prikazuje gdje CORS politika "živi" u našem lancu komunikacije između klijenta i poslužitelja:



> Slika 12: Ilustracija razmjene podataka između klijenta (Vue.js aplikacije) i poslužitelja (Express.js aplikacije) uz CORS politiku

Vratimo se na `pizza-express` projekt i instalirajmo `cors` paket:

```
→ cd app/pizza-express
→ npm install cors
```

`cors` paket nam omogućuje **jednostavnu konfiguraciju CORS politike na Express poslužitelju**. Uključit ćemo ga u glavnoj `index.js` datoteci poslužitelja.

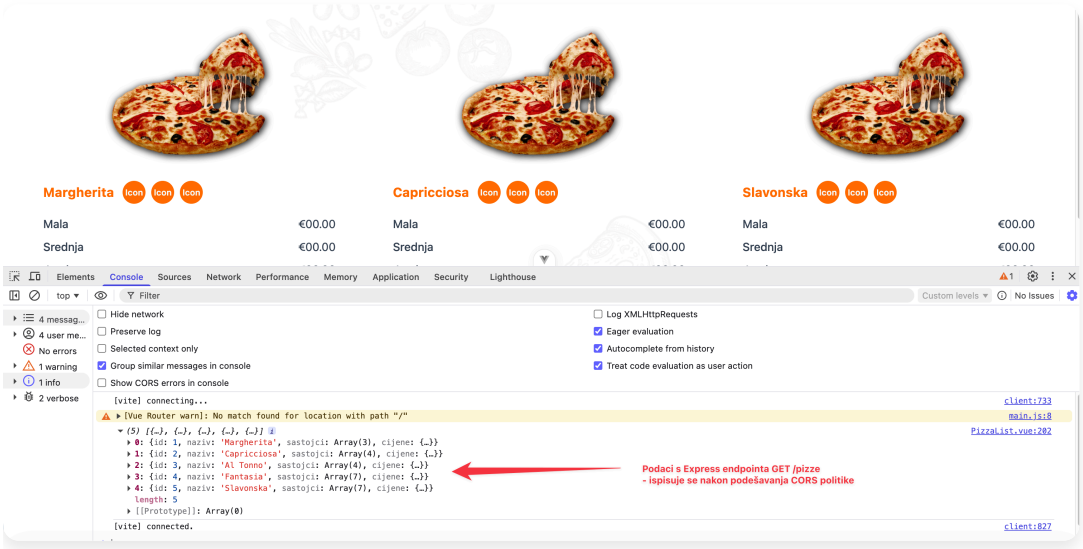
```
// app/pizza-express/index.js

import express from 'express';
import cors from 'cors'; // uvoz cors paketa
const app = express();
```

Dodajemo još jedan globalni *middleware* poziv, ovaj put za `cors`:

```
app.use(cors());
```

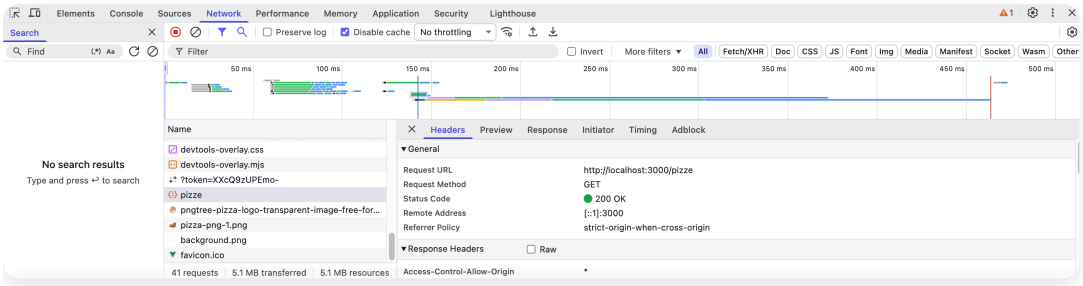
Na ovaj način smo dozvolili **svim domenama** da šalju zahtjeve našem Express poslužitelju. **Ovo nije dobro produkcijsko rješenje**, ali je u redu za razvojno okruženje i testiranje web aplikacije.

Vratite se na Vue.js aplikaciju i osvježite stranicu u pregledniku. Sada bi HTTP zahtjev trebao uspješno proći bez CORS greške, a podaci o pizzama trebali bi se ispisati u **konzoli preglednika**.



> Slika 13: Uspješan HTTP GET zahtjev s Vue.js aplikacije na Express poslužitelj nakon konfiguracije CORS politike (prikaz u konzoli preglednika)

Možete otvoriti i **Network tab** i pogledati detalje uspješnog zahtjeva.



> Slika 14: Detalji uspješnog HTTP GET zahtjeva s Vue.js aplikacije na Express poslužitelj nakon konfiguracije CORS politike na *backendu* (prikaz u Network tabu *developer toolsa*)

Ipak, na Express poslužitelju **poželjno je dodatno ograničiti CORS politiku** samo na domenu naše Vue.js aplikacije.

> **Važna napomena**: CORS je **dodatna sigurnosna značajka web preglednika** - ovo nije zamjena za adekvatnu autorizaciju i autentifikaciju HTTP zahtjeva, strogu validaciju i sanitizaciju podataka na *backend poslužitelju*, a HTTP zahtjeve je i dalje moguće poslati kroz HTTP klijente izvan preglednika (npr. Postman, curl, itd.) bez obzira na CORS postavke.

Možemo definirati CORS opcije prilikom poziva `cors()` funkcije:

```
const corsOptions = {
    origin: 'http://localhost:5173'
};
```

Ako je naša web aplikacija dostupna na više domena, možemo navesti i niz dozvoljenih domena:

```
const corsOptions = {
    origin: ['http://localhost:5173', 'http://example.com', 'http://mydomain.com']
};
```

- ovo može biti praktično ako imamo više *frontend* aplikacija koje trebaju pristupiti našem Express poslužitelju (npr. mobilna aplikacija i web aplikacija) ili
- u produkcijskom okruženju, **svakako navedite samo domene koje su vam potrebne**

Proslijedite ove opcije `cors()` *middleware* funkciju:

```
app.use(cors(corsOptions));
```

3.2 Dinamičko iscrtavanje podataka o pizzama (GET /pizze)

Sada kada smo uspostavili HTTP komunikaciju između Vue.js aplikacije i Express poslužitelja, možemo dinamički iscrtavati podatke o pizzama unutar `PizzaList.vue` komponente.

Navedeno možemo postići tako da pohranimo dohvaćene podatke o pizzama u **reaktivnu varijablu** i zatim koristimo Vue-ovu `v-for` direktivu za iteraciju kroz taj popis i iscrtavanje svake pizze.

Kako se HTTP zahtjev izvršava asinkrono, trebamo ga smjestiti unutar `onMounted` *lifecycle hooka* kako bismo bili sigurni da se HTTP zahtjev šalje tek nakon što je komponenta "montirana" u DOM strukturu.

Sintaksa:

```
// app/pizza-vue/src/components/PizzaList.vue

import { onMounted } from 'vue';

onMounted(() => {
    // kod koji se izvršava nakon što je Vue komponenta montirana
});
```

Primjer: prebacujemo rukovanje `axios` Promise objekta u `onMounted` *hook*

```
import { ref, onMounted } from 'vue';

const pizze = ref([]); // reaktivna varijabla za pohranu podataka o pizzama

onMounted(() => {
    axios
        .get('http://localhost:3000/pizze')
        .then(response => {
            pizze.value = response.data; // pohrana podataka o pizzama u reaktivnu varijablu
        })
        .catch(error => {
            console.error('Greška pri dohvaćanju podataka o pizzama:', error);
        });
});
console.log(pizze.value); // ispisuje podatke o pizzama (?)
```

Kod iznad neće raditi budući da je sinkroni, tj. `console.log(pizze.value);` će se izvršiti **prije nego što se HTTP zahtjev završi** i podaci budu pohranjeni u `pizze` varijablu. Iz tog razloga će se ispisati prazno polje `[]`.

Ako ga prebacimo ispod `pizze.value = response.data;` tj. unutar `.then()` metode, tada će se ispisati tek nakon što su podaci uspješno dohvaćeni.

```
.then(response => {
    pizze.value = response.data; // pohrana podataka o pizzama u reaktivnu varijablu
    console.log(pizze.value); // ispisuje podatke o pizzama nakon dohvaćanja HTTP odgovora
})
```

Ipak, kako bismo mogli ovaj kod "spakirati u funkciju", moramo koristiti `async/await` sintaksu unutar `onMounted` *hooka*:

```
onMounted(async () => {
    try {
        const response = await axios.get('http://localhost:3000/pizze');
        pizze.value = response.data; // pohrana podataka o pizzama u reaktivnu varijablu
        console.log(pizze.value); // ispisuje podatke o pizzama nakon dohvaćanja HTTP
odgovora
    } catch (error) {
        console.error('Greška pri dohvaćanju podataka o pizzama:', error);
    }
});
```

Logiku dohvaćanja praktično je izdvojiti u zasebnu asinkronu funkciju, npr. `fetchPizze()`, koju ćemo pozvati unutar `onMounted` *hooka*:

```
async function fetchPizze() {
    try {
        const response = await axios.get('http://localhost:3000/pizze'); // dodajemo await
kako bi sačekali odgovor asiknrone funkcije
        pizze.value = response.data; // pohrana podataka o pizzama u reaktivnu varijablu
        console.log(pizze.value); // ispisuje podatke o pizzama nakon dohvaćanja HTTP
odgovora
    } catch (error) {
        console.error('Greška pri dohvaćanju podataka o pizzama:', error);
    }
}
// u ovom slučaju onMounted ne treba biti async zato što ne koristimo await direktno unutar
njega
onMounted(() => {
    fetchPizze(); // pozivanje funkcije za dohvaćanje podataka o pizzama
});
```

Osvježite web aplikaciju i provjerite u konzoli preglednika dohvaćaju li se podaci o pizzama ispravno i jesu li pohranjeni u `pizze` reaktivnu varijablu.

3.2.1 `v-for` direktiva

Direktiva `v-for` nam omogućuje da iteriramo kroz polja ili objekte i iscrtavamo HTML elemente za svaki element u nizu ili svojstvo u objektu.

Prvi korak je identificirati **HTML strukturu koju želimo ponavljati za svaki element u polju**, odnosno želimo identificirati HTML elemente koji se **ponavljaju za svaku pizzu** (jedna kartica pizze).

U našem slučaju, to je početni `div` element kojem smo dodali `@click` direktivu:

```
<div
    @click="odaberiPizzu('Margherita')"
    :class="[
          'bg-inherit rounded-xl overflow-hidden cursor-pointer transition-all duration-300',
          odabrana_pizza === 'Capricciosa'
            ? 'ring-4 ring-orange-300 shadow-lg shadow-orange-300/50 scale-[1.02]'
            : 'hover:scale-[1.01]',
        ]"
>
    ...
</div>
```

Uočite dinamičke elemente unutar ovog `div`-a koje želimo zamijeniti s podacima iz reaktivne varijable `pizze`:

- **parametar** unutar `odaberiPizzu('pizza')` funkcije u `@click` eventu
- **uvjet** unutar `:class` direktive za isticanje odabrane pizze
- **naziv pizze**: `<h2 class="...">Pizza 1</h2>`
- **ponavljajuće ikone sastojaka** unutar `<div class="p-6">...</div>`
- **cijene za svaku veličinu pizze** unutar `<div class="space-y-2">...</div>`

> Napomena: Implementacija Vue `v-for` direktive je **konceptualno ekvivalentna** petlji `for ... of`, ali ovdje pišemo `for ... in` (ne zbuniti s JavaScript petljom koja iterira prema ključevima objekta!)

Sintaksa:

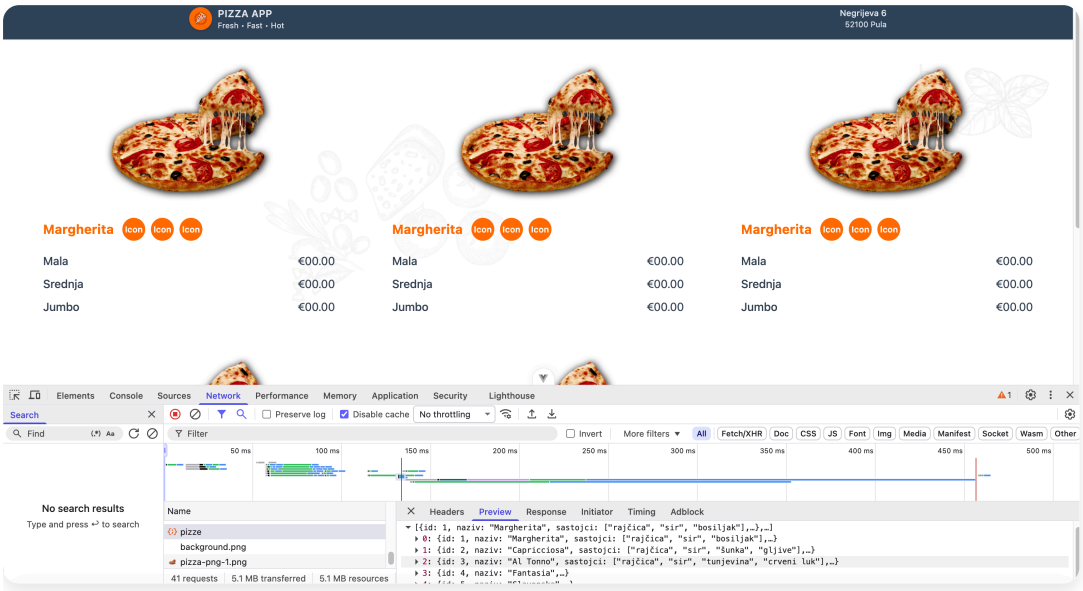
```
<div v-for="item in items" :key="item.id">
    <!-- sadržaj koji se ponavlja za svaki item -->
</div>
```

U našem slučaju, `item` predstavlja pojedinačnu pizzu iz niza `pizze`, pa možemo koristiti naziv `pizza` umjesto `item` radi bolje čitljivosti koda.

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->

<div
    v-for="pizza in pizze"
    :key="pizza.id"
    @click="odaberiPizzu(pizza.naziv)"
    :class="[
          'bg-inherit rounded-xl overflow-hidden cursor-pointer transition-all duration-300',
          odabrana_pizza === pizza.naziv
            ? 'ring-4 ring-orange-300 shadow-lg shadow-orange-300/50 scale-[1.02]'
            : 'hover:scale-[1.01]',
        ]"
    ...
></div>
```

Sada **obrišite preostale pizze** iz HTML predloška budući da će se one sada generirati dinamički pomoću `v-for` direktive na temelju podataka iz reaktivne varijable `pizze` napunjene podacima s poslužitelja.

Ako ste dobro implementirali `v-for` direktivu, sada biste trebali vidjeti ukupno pet jednakih pizza-kartica iscrtano u pregledniku, ali s pogrešnim podacima.



> Slika 15: Dodana `v-for` direktiva za dinamičko iscrtavanje pizza unutar `PizzaList.vue` komponente (prikaz u pregledniku)

Kako bismo izmijenili HTML sadržaj iterabilnom podatku iz `pizza` objekta, koristit ćemo **interpolaciju** (`{{ }}`), tj. Template Syntax.

Na primjer, za prikaz naziva pizze, zamijenit ćemo statički tekst `Margherita` s interpoliranom varijablom: `{{ pizza.naziv }}`:

```
<h2 class="text-lg font-bold text-orange-500 tracking-wide">{{pizza.naziv}}</h2>
```

Isto možemo napraviti za cijene budući da je svojstvo `pizza.cijene` objekt s ključevima: `mala`, `srednja`, i `jumbo`. Prema tome:

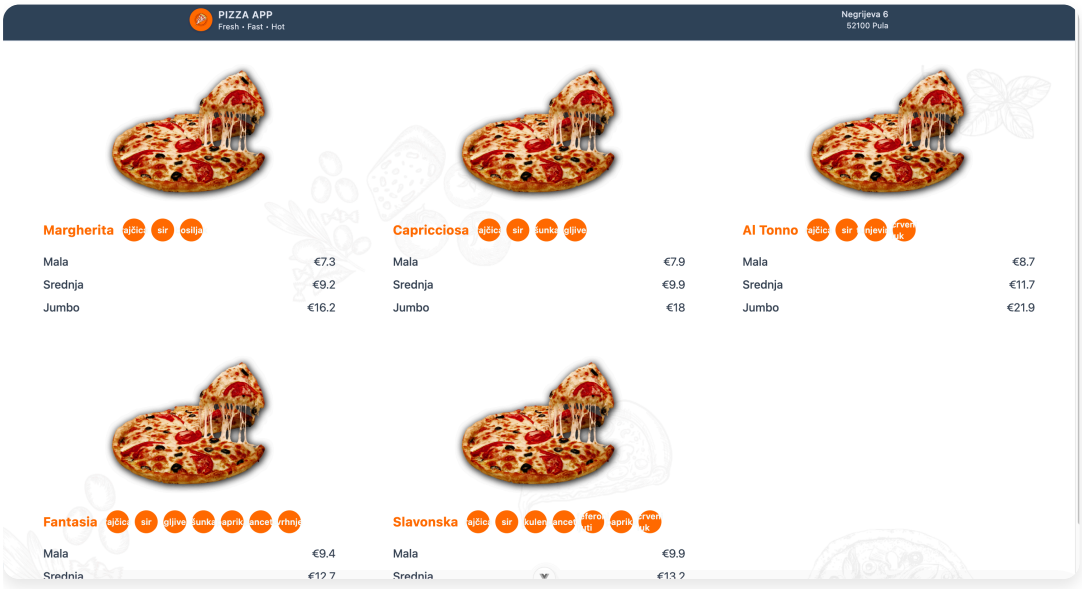
```
<div class="space-y-2">
    <div class="flex justify-between text-gray-700">
        <span class="font-medium">Mala</span>
        <span>€{{ pizza.cijene.mala }}</span>
    </div>

    <div class="flex justify-between text-gray-700">
        <span class="font-medium">Srednja</span>
        <span>€{{pizza.cijene.srednja}}</span>
    </div>

    <div class="flex justify-between text-gray-700">
        <span class="font-medium">Jumbo</span>
        <span>€{{pizza.cijene.jumbo}}</span>
    </div>
</div>
```

Želimo prikazati različite ikone ovisno o sastojcima svake pizze. Budući da kod sastojaka nema poretka (pohranjeni su u listi), idemo za početak samo izlistati naziv sastojaka unutar `<div class="p-6">...</div>`:

Kako je svojstvo `pizza.sastojci` polje stringova, moramo koristiti još jednu `v-for` direktivu za iteraciju kroz svaki sastojak.

Ovaj put ju dodajemo na ponavljajući element: `<div> class="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-center text-slate-50 font-semibold text-xs`.

```
<div class="flex space-x-2">
    <div v-for="for sastojak in pizze.sastojci" :key="sastojak" class="w-8 h-8 bg-orange-500
rounded-full flex items-center justify-center text-slate-50 font-semibold text-xs">Icon</div>
</div>
```



> Slika 16: Završena implementacija `v-for` direktive za dinamičko iscrtavanje podataka o pizzama unutar `PizzaList.vue` komponente (prikaz u pregledniku)

Svi podaci se uspješno iscrtavaju dinamički unutar `PizzaList.vue` komponente na temelju podataka dohvaćenih s Express poslužitelja 🚀.

Ipak, ne sviđa nam se kako se prikazuju sastojci - želimo vidjeti odgovarajuće ikone umjesto riječi "Icon". Idemo to implementirati.

3.2.2 Prikaz ikona sastojaka

Da bismo prikazali odgovarajuće ikone sastojaka, možemo definirati mapu (objekt) koja povezuje naziv sastojka s URL-om ikone ili lokalnom putanjom do slike.

Za ikone postoji mnoštvo besplatnih izvora na internetu, a mi ćemo koristiti Oh, Vue, Icons! biblioteku koja **agregira veliki broj besplatnih SVG ikona** iz različitih izvora.

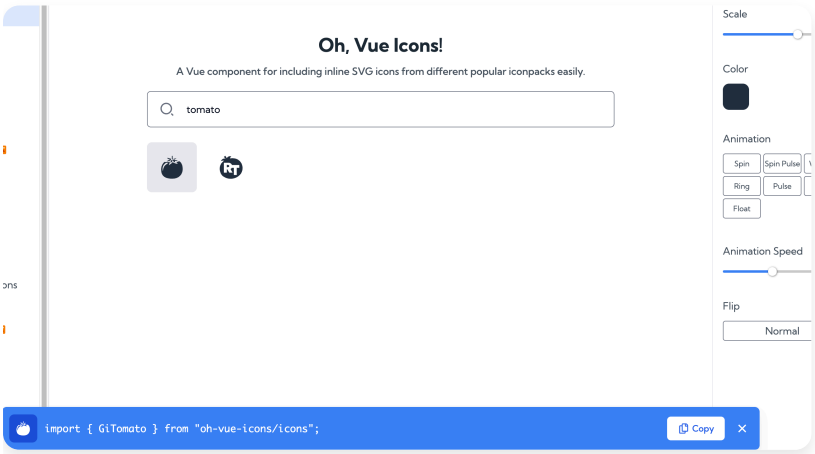
Instalirajte `oh-vue-icons` paket unutar Vue.js projekta:

```
→ cd /app/pizza-vue
→ npm install oh-vue-icons
```

Kako ova biblioteka nudi veliki broj ikona, dobra je praksa uvesti samo one koje ćemo koristiti kako bismo **smanjili veličinu konačnog JavaScript paketa** i **poboljšali performanse web aplikacije**.

Otvorite stranicu Oh, Vue, Icons! i potražite ikone koje odgovaraju sastojcima koje koristimo na pizzama.

Svakoj ikoni pridružen je jedinstveni identifikator koji ćemo koristiti za uvoz ikona, npr. za rajčicu može biti ikona `GiTomato`.



> Slika 17: Oh, Vue, Icons! web stranica za pregled i odabir ikona

Unutar `PizzaList.vue` komponente, uvezite potrebne ikone iz `oh-vue-icons` paketa:

```
import { GiTomato, GiCheeseWedge, GiSlicedMushroom, IoLeafSharp, CoHotjar, GiMilkCarton, GiBellPepper, LaPepperHotSolid, GiCannedFish, GiGarlic, FaBacon, GiHamShank } from 'oh-vue-icons/icons';
```

Zatim ćemo definirati jednostavnu mapu (objekt) koja povezuje naziv sastojka s odgovarajućim identifikatorom ikone (komponentom):

```
const ikoneSastojaka = {
    rajčica: GiTomato,
    sir: GiCheeseWedge,
    gljive: GiSlicedMushroom,
    bosiljak: IoLeafSharp,
    paprika: GiBellPepper,
    šunka: GiHamShank,
    'feferoni ljuti': LaPepperHotSolid,
    tunjevina: GiCannedFish,
    'crveni luk': GiGarlic, // nema ikone za luk :(
    panceta: FaBacon,
    kulen: CoHotjar,
    vrhnje: GiMilkCarton // nema ni ikone za vrhnje pa ćemo staviti mlijeko
};
```

Ikone iz biblioteke `oh-vue-icons` su Vue komponente, a iscrtavamo ih koristeći `v-icon` komponentu.

```
<v-icon :name="kebab-case-ikona" class="w-5 h-5" />
```

Problem je što imena ikona u `oh-vue-icons` biblioteci koriste `PascalCase` format (npr. `GiTomato`), dok `v-icon` komponenta očekuje `kebab-case` format (npr. `gi-tomato`).

Da bismo riješili ovaj problem, možemo definirati pomoćnu funkciju koja će pretvoriti `PascalCase` u `kebab-case` format **ili izmijeniti mapu** `ikoneSastojaka` da pohranjuje `kebab-case` nazive ikona kao vrijednosti.

> Napomena: `kebab-case` format koristi crtice za razdvajanje riječi, dok `PascalCase` format koristi velika slova za početak svake riječi bez razmaka ili crtica. Ovo nam je jednostavno ograničenje `oh-vue-icons` biblioteke koje moramo uzeti u obzir.

```
// mapa ikona sastojaka s kebab-case imenima
const ikoneSastojaka = {
    rajčica: 'gi-tomato',
    sir: 'gi-cheese-wedge',
    gljive: 'gi-sliced-mushroom',
    bosiljak: 'io-leaf-sharp',
    paprika: 'gi-bell-pepper',
    šunka: 'gi-ham-shank',
    'feferoni ljuti': 'la-pepper-hot-solid',
    tunjevina: 'gi-canned-fish',
    'crveni luk': 'gi-garlic',
    panceta: 'fa-bacon',
    kulen: 'co-hotjar',
    vrhnje: 'gi-milk-carton'
};
```

Almost there! Još moramo malo izmijeniti **sintaksu učitavanja ikona**, koristit ćemo funkciju `addIcons` iz `oh-vue-icons` paketa kako bismo registrirali ikone koje ćemo koristiti:

```
// app/pizza-vue/src/components/PizzaList.vue

import { addIcons } from 'oh-vue-icons';

// uvoz potrebnih ikona
import { GiTomato, GiCheeseWedge, GiSlicedMushroom, IoLeafSharp, CoHotjar, GiMilkCarton,
GiBellPepper, LaPepperHotSolid, GiCannedFish, GiGarlic, FaBacon, GiHamShank } from 'oh-vue-
icons/icons';

// registracija ikona koje ćemo koristiti
addIcons(GiTomato, GiCheeseWedge, GiSlicedMushroom, IoLeafSharp, GiBellPepper, GiHamShank,
LaPepperHotSolid, GiCannedFish, GiGarlic, FaBacon, CoHotjar, GiMilkCarton);
```

> Napomena: Moramo registrirati samo one ikone koje ćemo koristiti. Više o tome u dokumentaciji Oh, Vue, Icons!. Ovo je vrlo važno za optimizaciju web stranice - **ne želimo učitavati na tisuće ikona u Vue aplikaciju** ako ćemo koristiti samo nekoliko njih.

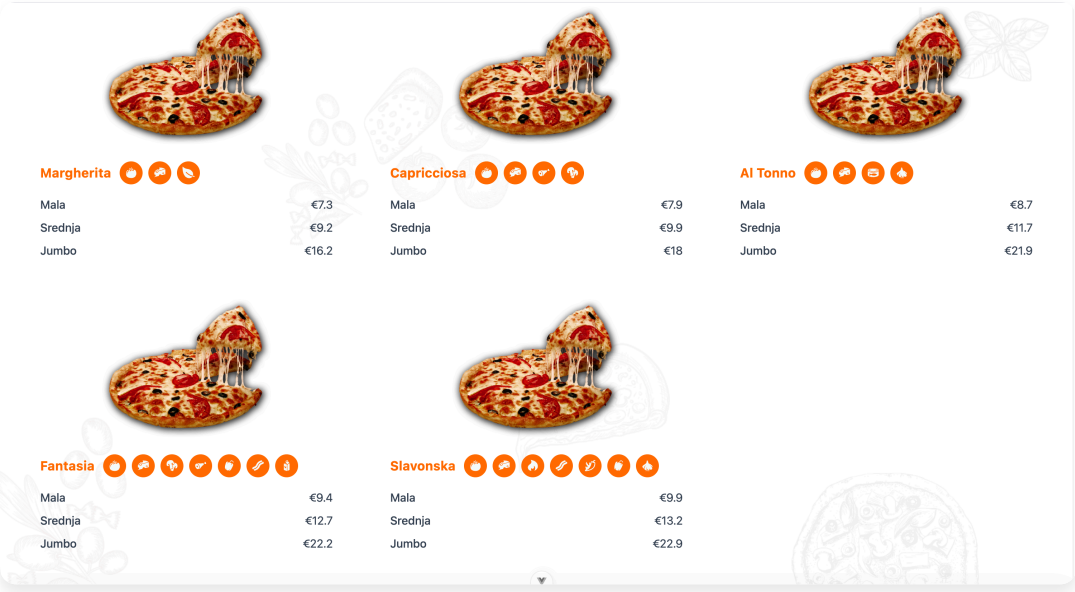
Za kraj, moramo registrirati `OhVueIcons` *plugin* unutar glavne `main.js` datoteke Vue.js projekta:

```
// app/pizza-vue/src/main.js

import OhVueIcon from 'oh-vue-icons';

app.component('v-icon', OhVueIcon); // mapiraj OhVueIcon komponentu na "v-icon" HTML tag
```

Sada možemo iscrtati ikone sastojaka unutar `v-for` direktive u `PizzaList.vue` komponenti:

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->

  <div
    v-for="sastojak in pizza.sastojci"
    :key="sastojak"
    class="w-8 h-8 bg-orange-500 rounded-full flex items-center justify-center text-slate-50
font-semibold text-xs"
  >
    <v-icon :name="ikoneSastojaka[sastojak]" />
  </div>
</div>
```

To je to! Ispravno smo prikazali sve podatke s poslužitelja, uključujući i ikone sastojaka za svaku pizzu.



> Slika 18: Završena implementacija prikaza ikona sastojaka unutar `PizzaList.vue` komponente (prikaz u pregledniku)

3.2.3 Dodavanje javnih slika na poslužitelj

Kako nam ne bi svaka pizza imala istu sliku, možemo dodati prave slike pizza u podatke na Express poslužitelju te ih potom prikazati unutar Vue.js aplikacije.

Za sada nećemo učitavati slike, već ćemo iskoristiti "javno dostupne" slike s interneta. Izvor koji će se koristiti u ovoj skripti su slike s weba Pulske pizzerije TiVoli.

Unutar `pizza-express/data/data.js` datoteke, dodajte ključ `slika_url` za svaku sliku te postavite odgovarajući **javni URL** slike s interneta.

Primjer:

```
// app/pizza-express/data/data.js

{id: 1, naziv: "Margherita", sastojci: ["rajčica", "sir", "bosiljak"], cijene: {"mala": 7.30,
"srednja": 9.20, "jumbo": 16.20},
slika_url:"https://pizzeria-tivoli.com.hr/uploads/pizza-margherita-u6kflo.jpg"},

ostale pizze ...
```

Provjerite na Postmanu da li se novi podaci ispravno vraćaju s poslužitelja. **Ako ne radi u Postmanu, neće raditi ni u Vue.js aplikaciji**.

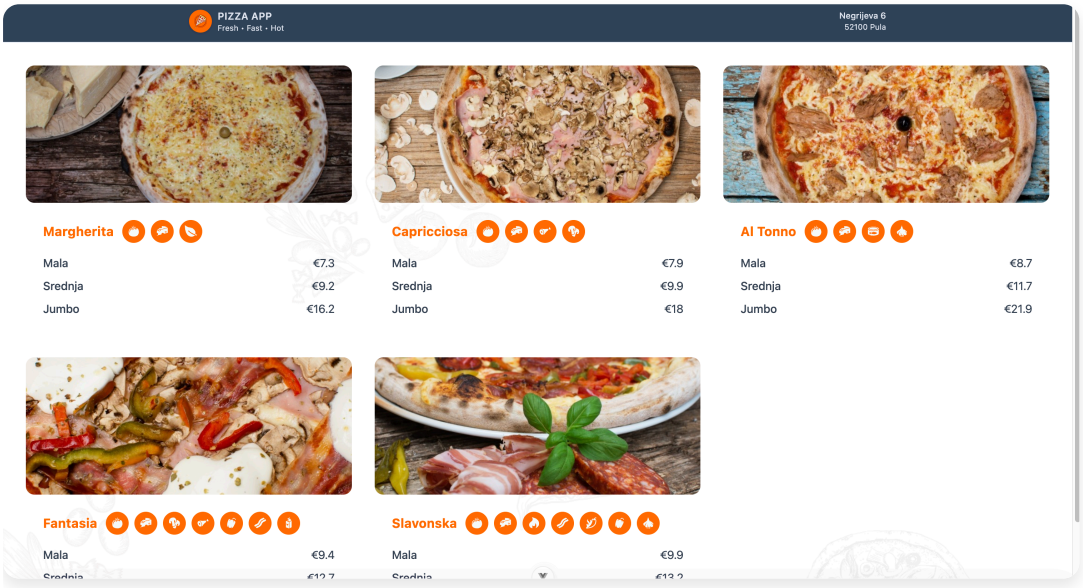
Vratite se na Vue.js aplikaciju i unutar `PizzaList.vue` komponente, iscrtajte sliku unutar glavnog `div`-a za svaku pizzu:

Koristimo dinamičko svojstvo `:src` za postavljanje URL-a slike te `:alt` za dinamički alternativni tekst slike (možemo kombinirati ternarni operator kao što smo radili s dinamičkim klasama):

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->

<img :src="pizza.slika_url" :alt="pizza.naziv" class="w-full h-full object-contain" />
```

Malo ćemo izmijeniti stilove kako bi slika zauzela cijeli kontejner i kako bi malo zaoblili rubove, obzirom da više nemamo slike s transparentnom pozadinom.

```
<div class="w-full h-48 flex items-center justify-center bg-inherit overflow-hidden rounded-xl">
    <img :src="pizza.slika_url" :alt="pizza.naziv" class="w-full h-full object-cover" />
</div>
```



> Slika 19: Završena implementacija prikaza slika pizza unutar `PizzaList.vue` komponente (prikaz u pregledniku)

3.3 Slanje nove narudžbe (POST /narudzbe)

Implementirat ćemo funkcionalnost slanja nove narudžbe na Express poslužitelj koristeći definirani `POST /narudzbe` endpoint poslužitelja. Međutim, idemo prvo uključiti UI element za odabir pizza, veličina i količina unutar `PizzaList.vue` komponente.

Odabirom određene pizze, želimo prikazati *footer-izbornik* na dnu ekrana gdje korisnik može odabrati veličinu pizze te količinu. Izbornik se mora prikazati odabirom pizze iz `PizzaList`.

Dodat ćemo novu komponentu `OrderFooter.vue` unutar `components` direktorija Vue.js projekta.

```
→ cd app/pizza-vue/src/components
→ touch OrderFooter.vue
```

Strukturu i stil komponente možete preuzeti iz gotovih predložaka (*vue-templates*) ili možete implementirati sami prema vlastitim željama.

Ovoj komponenti želimo proslijediti podatke o odabranoj pizzi, pa ćemo definirati `props` (*properties*) unutar `OrderFooter.vue` komponente.

U Vue.js, `props` su način na koji roditeljska (*eng. parent*) komponenta može prosljeđivati podatke svojoj djeci (*eng. child* komponenatama). Djeca komponente predstavljaju komponente koje su ugniježdene (iscrtavaju se) unutar roditeljske komponente. Ovo je korisno raditi kako bismo odvojili **ponavljajuće UI elemente**. Konkretno, `OrderFooter.vue` komponenta će biti dijete `PizzaList.vue` komponente.

U Vue 3, definiramo `props` unutar `<script setup>` bloka koristeći `defineProps` funkciju.

Sintaksa:

```
const props = defineProps({
    naziv_propa: tip_podatka,
    drugi_prop: tip_podatka,
    ...
});
```

- moguće je proslijediti bilo koji JavaScript tip kao `prop`

Proslijedit ćemo komponenti objekt odabrane pizze koji sadrži sve potrebne informacije o odabranoj pizzi (naziv, cijene, sastojci, itd.).

```
const props = defineProps({
    odabranaPizza: {
        type: Object,
        required: true // označava da je ovaj prop obavezan
    }
});
```

Uočite dinamičke elemente unutar `OrderFooter.vue` komponente koje moramo zamijeniti s podacima iz `odabranaPizza` propa:

```vue
<!-- app/pizza-vue/src/components/OrderFooter.vue -->

<!--slika pizze-->
<img :src="url_slike_ovdje" alt="slika ovdje" class="w-12 h-12 sm:w-16 sm:h-16 rounded-lg object-cover shadow-md shadow-black/40" />

...
<!--naziv pizze-->
<h3 class="font-bold tracking-wide text-base sm:text-lg text-orange-400">{{ naziv_pizze_ovdje }}</h3>

...

<!--veličine i cijene-->
<button
    v-for="velicina in dostupneVelicine"
    :key="size.label"
    class="px-3 py-1 cursor-pointer rounded-lg border border-slate-500 bg-slate-600/40 hover:bg-orange-500 hover:border-orange-400 hover:text-white transition-all text-sm sm:text-base"
>
    {{ velicina.oznaka }} — {{ velicina.cijena }}
</button>
```

Jedino što nas može zbuniti je: kako iterirati ispravno kroz veličine i cijene pizze budući da je svojstvo `cijene` objekt, a ne polje?

Jednostavno očekujemo dvije varijable `(value, key)` kao lokalni parametar unutar `v-for` direktive:

Sintaksa:

```vue
<!--Pripazite! Nije (key, value) već (value, key) kod iteracije objekta-->
<div v-for="(value, key) in object" :key="key">
    <!-- sadržaj koji koristi value i key -->
</div>
```

U našem slučaju, `key` predstavlja veličinu pizze (`mala`, `srednja`, `jumbo`), dok `value` predstavlja cijenu za tu veličinu. Dakle, radimo sljedeće:

```
<div class="flex items-center justify-center sm:justify-start flex-wrap gap-2 w-full sm:w-auto">
    <button
        v-for="(cijena, velicina) in odabranaPizza.cijene"
        :key="velicina"
        class="px-3 py-1 cursor-pointer rounded-lg border border-slate-500 bg-slate-600/40 hover:bg-orange-500 hover:border-orange-400 hover:text-white transition-all text-sm sm:text-base"
    >
        {{ velicina }} – {{ cijena }}€
    </button>
</div>
```

Sada možemo koristiti `OrderFooter.vue` komponentu unutar `PizzaList.vue` komponente i proslijediti odabranu pizzu kao `prop`.

Učitaje `OrderFooter.vue` komponentu unutar `PizzaList.vue` komponente:

```
// app/pizza-vue/src/components/PizzaList.vue

import OrderFooter from './OrderFooter.vue';
```

Dodat ćemo komponentu na dnu `PizzaList.vue` predloška, ispod glavnog `div`-a koji sadrži popis pizza. Ipak, prije toga želimo u reaktivnu varijablu `odabranaPizza` pohraniti objekt odabrane pizze (umjesto samo naziva pizze kao do sada).

```
// app/pizza-vue/src/components/PizzaList.vue

function odaberiPizzu(pizza) {
    odabrana_pizza.value = pizza; // pohranjujemo cijeli objekt pizze
    console.log('Odabrana pizza:', pizza);
}
```

Reaktivnu varijablu možemo jednostavno ažurirati kod `@click` direktive:

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->

<div
    v-for="pizza in pizze"
    :key="pizza.id"
    @click="odaberiPizzu(pizza)" <!-- prosljeđujemo cijeli objekt pizze -->
    ...
></div>
```

Sada ćemo dodati komponentu, ali ćemo ju prikazati samo ako je neka pizza odabrana (kada `odabrana_pizza` nije `null`):

Za to koristimo `v-if` direktivu:

Sintaksa:

```
<!-- Iscrtava komponentu samo ako je uvjet istinit -->
<ChildComponent v-if="uvjet" :prop1="vrijednost1" :prop2="vrijednost2" />
```

Dakle: dodajemo nakon zadnjeg `</div>` unutar `PizzaList.vue` predloška:

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->
...
  </div>
  <OrderFooter v-if="odabrana_pizza" :odabrana-pizza="odabrana_pizza" /> <!-- ako je odabrana pizza, prikaži OrderFooter komponentu i proslijedit taj objekt -->
</template>
```

Također, sada moramo ažurirati dinamičku klasu kako ne bi izgubili *fancy pizza highlight* efekt na odabranoj pizzi:

```
odabrana_pizza.naziv === pizza.naziv
```

Ipak, nakon ove promjene i osvježavanja aplikacije dobit ćemo grešku u konzoli.

```
Uncaught (in promise) TypeError: Cannot read properties of null (reading 'naziv')
```

Ova greška se događa zato što je početna vrijednost `odabrana_pizza` varijable `null`, pa kada Vue pokuša pristupiti `odabrana_pizza.naziv` prije nego što je neka pizza odabrana, javlja se greška.

Opcionalno lančanje (Optional Chaining)

U Vue3, problem je moguće riješiti vrlo jednostavno koristeći JavaScript opcionalno lančanje (*eng. optional chaining*) operatorom `?.`.

Operator `?.` omogućuje sigurno pristupanje svojstvima objekta koji može biti `null` ili `undefined`. Ako je objekt `null` ili `undefined`, izraz će se odmah evaluirati na `undefined` umjesto da baci grešku.

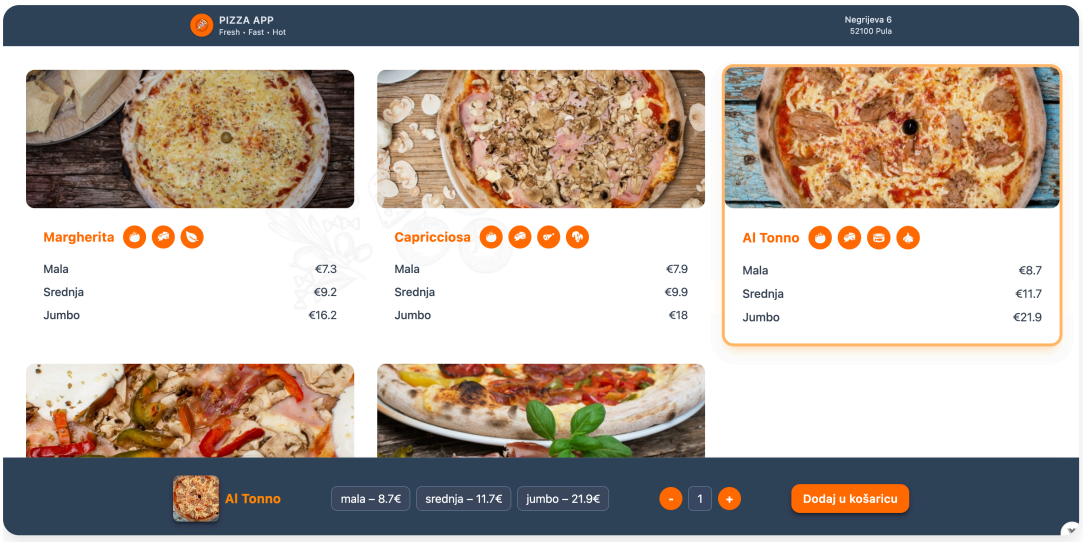
Sintaksa:

```
objekt?.svojstvo;

// ekvivalentno
objekt == null ? undefined : objekt.svojstvo; // ako je objekt null, vrati undefined, inači vrati svojstvo objekta
```

Dakle, možemo izmijeniti uvjet unutar `:class` direktive na sljedeći način:

```
<!-- app/pizza-vue/src/components/PizzaList.vue -->
odabrana_pizza?.naziv === pizza.naziv
```

To je to! Sada kada odaberemo pizzu iz popisa, trebali bismo vidjeti *footer-izbornik* s podacima o odabranoj pizzi.



> Slika 20: Dodana `OrderFooter.vue` komponenta unutar `PizzaList.vue` komponente (prikaz u pregledniku)

Emitiranje događaja (Event Emitting)

Dodat ćemo još dva načina za zatvaranje *footer-izbornika*, tj. poništavanje odabira pizze:

1. način: klikom na gumb `X` unutar *footer-izbornika*
2. način: klikom na `ESC` tipku na tipkovnici

1. način:

Unutar `OrderFooter.vue` komponente, dodajemo prvo `X` u gornji desni kut *footer-izbornika*:

- dodaje kao prvi element nakon `footer` taga

```
<!-- app/pizza-vue/src/components/OrderFooter.vue -->

<button class="absolute top-2 right-2 text-slate-300 hover:text-white text-xl font-bold cursor-pointer">×</button>
```

E sad, rekli samo da ako želimo proslijediti podatke iz roditeljske komponente u dječju komponentu, možemo koristiti `props`, ali kako ćemo proslijediti podatke (ili signal) iz dječje komponente natrag u roditeljsku komponentu? Tj. kako ćemo reći `PizzaList.vue` komponenti da je korisnik kliknuo na `X` gumb unutar `OrderFooter.vue` komponente i da može poništiti odabir pizze?

Za to koristimo **emitiranje događaja** (*eng. event emitting*) u Vue.js.

Sintaksa:

```
const emit = defineEmits(['naziv_dogadaja']);
```

Naziv događaja može biti bilo koji string, ali je dobra praksa koristiti opisne nazive koji jasno ukazuju na svrhu događaja. Naziv našeg događaja bit će `close`.

```
// app/pizza-vue/src/components/OrderFooter.vue
const emit = defineEmits(['close']);
```

Zatim, unutar `@click` eventa gumba `X`, pozivamo `emit(emit_naziv)` funkciju kako bismo emitirali `close` događaj:

```
<button class="absolute top-2 right-2 text-slate-300 hover:text-white text-xl font-bold cursor-pointer" @click="emit('close')">×</button>
```

Emit možemo dohvatiti iz roditeljskog *hooka* na isti način kao što bismo pozivali direktivu, poput `@click`.

Dodajemo `@close` direktivu na `OrderFooter` komponentu unutar `PizzaList.vue` komponente te što se dešava kada se dogodi `close` događaj:

- `@close="odabrana_pizza = null"` - poništavamo odabir pizze postavljanjem `odabrana_pizza` varijable na `null` jednom kad se dogodi `close` emit

```
<OrderFooter v-if="odabrana_pizza" :odabrana-pizza="odabrana_pizza" @close="odabrana_pizza = null" />
```

2. način:

Za hvatanje `ESC` tipke na tipkovnici, možemo koristiti `window` objekt za dodavanje globalnog event listenera unutar `onMounted` *hooka* u `PizzaList.vue` komponenti.

```
// app/pizza-vue/src/components/PizzaList.vue

onMounted(() => {
    window.addEventListener('keydown', event => {
        if (event.key === 'Escape') {
            odabrana_pizza.value = null; // poništavamo odabir pizze
        }
    });
});
```

To je to! Sada možemo zatvoriti *footer-izbornik* klikom na `X` gumb ili pritiskom na `ESC` tipku na tipkovnici 😎.

Implementacija preostalih UI funkcionalnosti

Sljedeći korak je **evidentirati stavke narudžbe** jednom kad korisnik klikne na gumb `Dodaj u košaricu` unutar `OrderFooter.vue` komponente.

Idemo unaprijediti korisničko sučelje dodavanjem sljedećih funkcionalnosti:

1. Jasno istaknut odabir veličine pizze.
2. Gumbi `+` i `–` za povećanje ili smanjenje količine naručenih pizza.
3. Automatski izračun i prikaz cijene prema odabranoj veličini i količini.
4. Gumb **Dodaj u košaricu** zajedno s prikazom dodane stavke.
5. Dodavanje i implementacija završnog gumba **Pošalji narudžbu**.

Krenimo redom!

Na Express poslužitelju smo definirali strukturu narudžbe koja izgleda ovako:

```
{
    "narucene_pizze": [
        {
            "naziv": "Capricciosa",
            "velicina": "mala",
            "kolicina": 3
        },
        {
            "naziv": "Slavonska",
            "velicina": "srednja",
            "kolicina": 2
        }
    ],
    "podaci_dostava": {
        "prezime": "Pilić",
        "adresa": "Ilica 305, Zagreb",
        "telefon": "091234567"
    }
}
```

Unutar `OrderFooter.vue` komponente, definirat ćemo reaktivne varijable za pohranu `narucene_pizze` (polje naručenih pizza) i popratnu funkciju `dodajUNarudzbu` koja će se pozivati klikom na gumb **Dodaj u košaricu**.

```
// app/pizza-vue/src/components/OrderFooter.vue

const narucene_pizze = ref([]);

function dodajUNarudzbu() {
    // logika za dodavanje pizze u narudžbu
}
```

Odabir veličine pizze možemo postići reaktivnim tailwind klasama (UI) kao što smo već radili za odabir pizze.

Definirat ćemo reaktivnu varijablu `odabranaVelicina` koja će pohranjivati trenutno odabranu veličinu pizze. Uz to, možemo postaviti i početnu naručenu količinu na `1`.

```
const odabranaVelicina = ref('mala'); // početna (zadana) vrijednost
const kolicina = ref(1); // početna (zadana) količina
```

Idemo pregledati `button` HTML strukturu:

```
<div class="flex items-center justify-center sm:justify-start flex-wrap gap-2 w-full sm:w-auto">
    <button
        v-for="(cijena, velicina) in odabranaPizza.cijene"
        :key="velicina"
        class="px-3 py-1 cursor-pointer rounded-lg border border-slate-500 bg-slate-600/40 hover:bg-orange-500 hover:border-orange-400 hover:text-white transition-all text-sm sm:text-base"
    >
        {{ velicina }} – {{ cijena }}€
    </button>
</div>
```

Tailwind klasa `bg-slate-600/40` definira pozadinsku boju gumba. Možemo ju izmijeniti u `bg-orange-500` kada je veličina odabrana (malo snažnija boja od hover efekta).

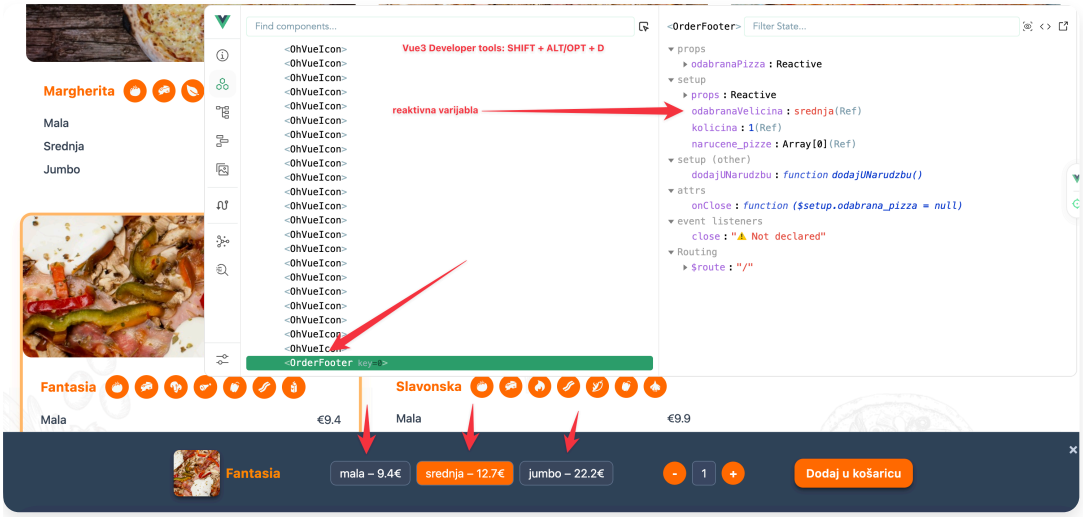
Dodajemo dinamičku klasu unutar `:class` direktive:

```
<button
    v-for="(cijena, velicina) in odabranaPizza.cijene"
    :key="velicina"
    :class="[
            'px-3 py-1 rounded-lg border border-slate-500 text-sm sm:text-base hover:bg-orange-500 hover:text-white transition-all cursor-pointer',
            odabranaVelicina === velicina
              ? 'bg-orange-500 text-white'
              : 'bg-slate-600/40 text-white',
          ]"
>
    {{ velicina }} – {{ cijena }}€
</button>
```

Još moramo dodati `@click` event na gumb kako bismo ažurirali `odabranaVelicina` varijablu:

```
<button ... @click="odabranaVelicina = velicina">{{ velicina }} – {{ cijena }}€</button>
```

Otvorite *Vue Devtools* u pregledniku i provjerite mijenja li se reaktivna varijabla `odabranaVelicina` unutar `OrderFooter.vue` komponente kada kliknete na različite veličine pizza.



> Slika 21: Reaktivna varijabla `odabranaVelicina` unutar `OrderFooter.vue` komponente (prikaz u Vue Devtools)

Idemo dalje!

Sličan pristup koristit ćemo za implementaciju gumba `+` i `–` za povećanje i smanjenje količine naručenih pizza.

Prvo moramo hardkodiranu jedinicu `1` unutar HTML strukture zamijeniti s reaktivnom varijablom `kolicina`.

```
<!-- app/pizza-vue/src/components/OrderFooter.vue -->

<div class="px-3 py-1 bg-slate-600/40 backdrop-blur-sm rounded-md border border-slate-500 text-sm sm:text-base">{{ kolicina }}</div>
```

`+` i `-` su nam gumbi kojima možemo dodati `@click` direktive za ažuriranje reaktivne varijable `kolicina`:

```
<!-- gumb za smanjenje količine -->
<button
    @click="kolicina > 1 ? kolicina-- : kolicina = 1"
    class="w-8 h-8 flex items-center justify-center rounded-full bg-orange-500 text-white
font-bold hover:bg-orange-600 transition-all cursor-pointer"
>
    -
</button>

<!-- gumb za povećanje količine -->
<!-- ternarni izraz ovdje nije obavezan, ali može ostati -->
<button
    @click="kolicina ? kolicina++ : (kolicina = 1)"
    class="w-8 h-8 flex items-center justify-center rounded-full bg-orange-500 text-white
font-bold hover:bg-orange-600 transition-all cursor-pointer"
>
    +
</button>
```

Izraz: `kolicina > 1 ? kolicina++ : (kolicina = 1)` osigurava da količina nikada ne padne ispod `1` te je ekvivalentan sljedećem:

> Hint: JavaScript Ternarni operator (`?`) se često koristi u Vue.js aplikacijama kada želimo unutar direktiva ili drugih template izraza napisati kratki logički uvjet.

```
if (kolicina > 1) {
    kolicina--;
} else {
    kolicina = 1;
}
```

Prije implementacije logike za dodavanje stavke u narudžbu, želimo prikazati **automatski izračunatu cijenu** prema odabranoj veličini i količini. Dodat ćemo cijenu **između odabrane količine** i gumba **Dodaj u košaricu**.

```
<div class="w-full sm:w-auto text-center font-semibold text-lg text-orange-400 tracking-
wide">Ukupno: {{ ukupna_cijena_stavke }}€</div>
```

Za izračun ukupne cijene stavke, praktično je koristiti Vue 3 computed properties unutar `<script setup>` bloka.

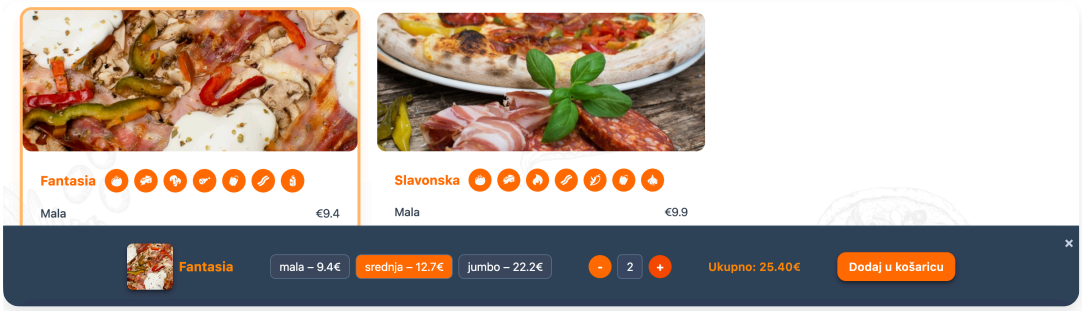
Zašto computed property? Zašto ne običnu reaktivnu varijablu ili funkciju?

- ne možemo koristiti običnu reaktivnu varijablu jer se vrijednost mora ažurirati svaki put kada se promijeni `odabranaVelicina` ili `kolicina`
- običnu funkciju bismo mogli koristiti, međutim computed property je puno bolje i optimiziraje rješenje jer Vue *cache*-ira vrijednost dok se ne promijene ovisnosti (u našem slučaju `odabranaVelicina` i `kolicina`)

`computed` svojstvo moramo uključiti iz `vue` paketa:

```
// app/pizza-vue/src/components/OrderFooter.vue

import { computed } from 'vue';

const ukupna_cijena_stavke = computed(() => {
    const cijenaPoKomadu = props.odabranaPizza.cijene[odabranaVelicina.value];
    return (cijenaPoKomadu * kolicina.value).toFixed(2);
});
```

```
<div class="w-full sm:w-auto text-center font-semibold text-lg text-orange-400 tracking-wide">Ukupno: {{ ukupna_cijena_stavke || '0.00' }}€</div>
```



> Slika 22: Prikaz automatski izračunate cijene unutar `OrderFooter.vue` komponente (prikaz u pregledniku) s interaktivnim odabirom veličine i količine

---

Sada možemo implementirati funkcionalnost dodavanja stavke u narudžbu klikom na gumb **Dodaj u košaricu**.

Unutar `dodajUNarudzbu` funkcije, kreiramo novi objekt stavke narudžbe koji sadrži naziv pizze, odabranu veličinu i količinu (onako kako očekuje Express poslužitelj):

```
// app/pizza-vue/src/components/OrderFooter.vue

function dodajUNarudzbu() {
    const novaStavka = {
        naziv: props.odabranaPizza.naziv,
        velicina: odabranaVelicina.value,
        kolicina: kolicina.value
    };
    narucene_pizze.value.push(novaStavka); // dodajemo stavku u polje naručenih pizza
    console.log('Naručene pizze:', narucene_pizze.value);
}
```

Pozivamo `dodajUNarudzbu` funkciju klikom na gumb **Dodaj u košaricu**:

```
<button
    @click="dodajUNarudzbu"
    class="bg-orange-500 text-white font-semibold px-4 py-2 rounded-xl shadow-md shadow-black/40 hover:bg-orange-600 transition-all tracking-wide cursor-pointer w-full sm:w-auto text-center"
>
    Dodaj u košaricu
</button>
```

Provjerite u konzoli preglednika i *Vue Devtools* da li se stavke ispravno dodaju u `narucene_pizze` polje kada kliknete na gumb **Dodaj u košaricu**.

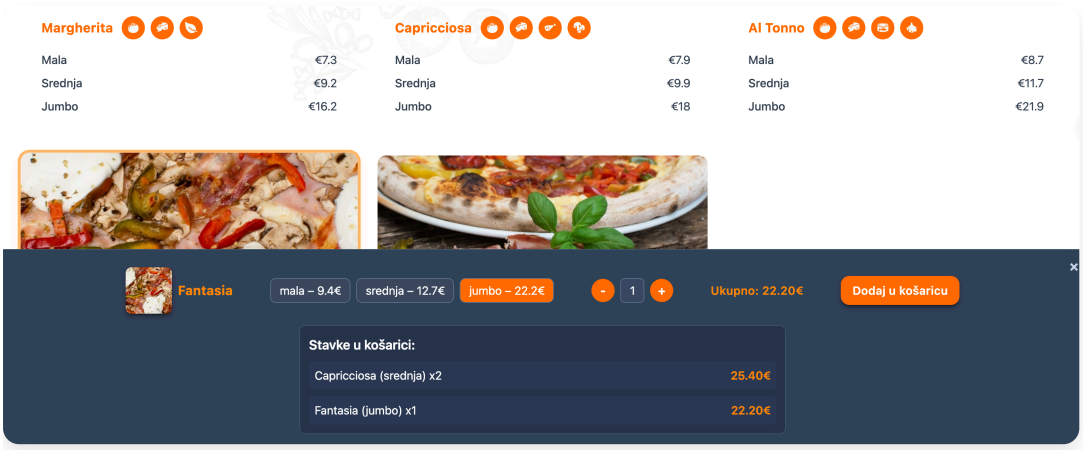
Kako bismo uvjerili i samog korisnika aplikacije, možemo dodavati mali graditi prikaz stavki:

Možete dodati sljedeći HTML isječak prije zatvaranja `footer` taga unutar `OrderFooter.vue` komponente:

- sljedeći Vue isječak nema ništa posebno novo, već samo iscrtava stavke iz `narucene_pizze` polja koristeći `v-for` direktivu s ključevima i vrijednostima iz objekta

```
<!-- app/pizza-vue/src/components/OrderFooter.vue -->
 ...
    <div
      v-if="narucene_pizze.length"
      class="mt-4 max-w-2xl mx-auto max-h-40 overflow-y-auto bg-slate-800/50 backdrop-blur-sm rounded-lg p-3 border border-slate-600"
    >
      <h4 class="font-semibold text-lg text-white mb-2">Stavke u košarici:</h4>
      <ul class="space-y-2">
        <li
          v-for="(stavka, index) in narucene_pizze"
          :key="index"
          class="flex items-center justify-between bg-slate-700/50 rounded-md p-2"
        >
          <div class="text-white">
            {{ stavka.naziv }} ({{ stavka.velicina }}) x{{ stavka.kolicina }}
          </div>
          <div class="text-orange-400 font-semibold">
            {{ (props.odabranaPizza.cijene[stavka.velicina] * stavka.kolicina).toFixed(2) }}€
          </div>
        </li>
      </ul>
    </div>
  </footer>
```

Testirajmo aplikaciju! Dodat ćemo 2 srednje Capricciose i 1 jumbo Fantasiu u košaricu.



> Slika 23: Prikaz stavki unutar `OrderFooter.vue` komponente (prikaz u pregledniku)

Napokon, dodajmo gumb **Naruči** unutar `OrderFooter.vue` komponente koji će poslati `POST /narudzbe` zahtjev na Express poslužitelj s podacima o narudžbi.

Dodat ćemo gumb pored gumba **Dodaj u košaricu**:

- klikom na gumb, pozvat ćemo `posaljiNarudzbu` funkciju koja će sadržavati logiku slanja narudžbe na Express poslužitelj

```
<button
...
    Dodaj u košaricu
</button>

<button
    @click="posaljiNarudzbu"
    class="bg-orange-500 text-white font-semibold px-4 py-2 rounded-xl shadow-md shadow-black/40 hover:bg-orange-600 transition-all tracking-wide cursor-pointer w-full sm:w-auto text-center"
>
    Naruči
</button>
```

Idemo implementirati `posaljiNarudzbu` funkciju unutar `<script setup>` bloka `OrderFooter.vue` komponente.

Nismo nigdje od korisnika prikupili podatke za dostavu (prezime, adresa, telefon), pa ćemo ih hardkodirati unutar funkcije za sada (ostaje za zadaću).

Moramo poslati zahtjev na `http://localhost:3000/narudzbe` s JSON tijelom koje sadrži polje `narucene_pizze` i objekt `podaci_dostava`.

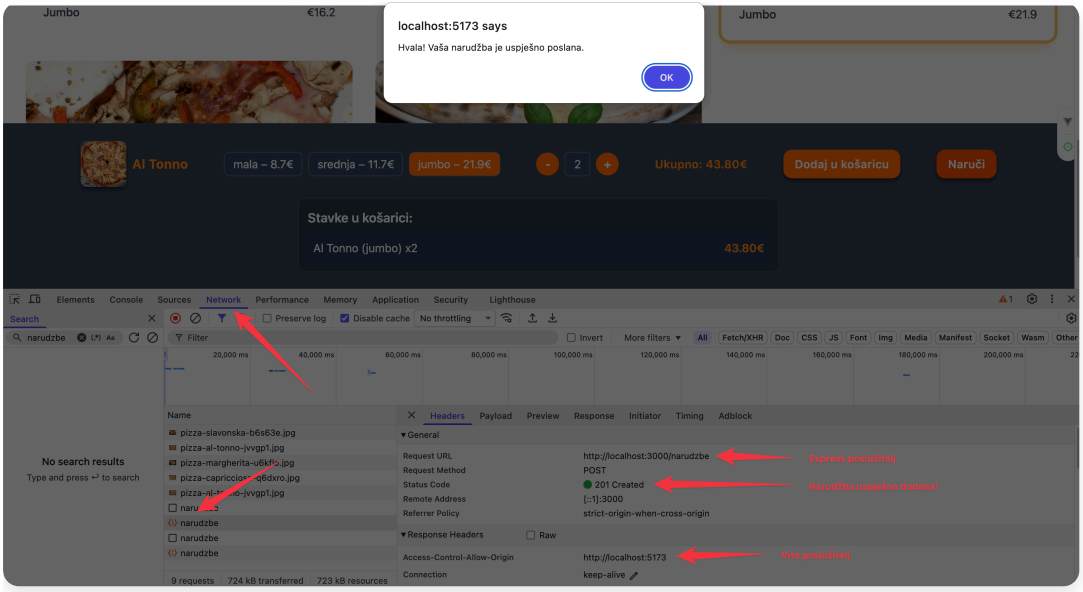
Definirat ćemo asinkronu funkciju `posaljiNarudzbu` koristeći `axios` za slanje `POST` zahtjeva:

```
// app/pizza-vue/src/components/OrderFooter.vue

import axios from 'axios';

async function posaljiNarudzbu() {
    try {
        // alert ako je košarica prazna
        if (narucene_pizze.value.length === 0) {
            alert('Košarica je prazna! Molimo dodajte pizze prije narudžbe.');
            return;
        }

        // hardkodirani podaci za dostavu
        const podaciZaDostavu = {
            prezime: 'Pilić',
            adresa: 'Ilica 305, Zagreb',
            telefon: '091234567'
        };

        const odgovor = await axios.post('http://localhost:3000/narudzbe', {
            narucene_pizze: narucene_pizze.value,
            podaci_dostava: podaciZaDostavu
        });

        console.log('Narudžba uspješno poslana:', odgovor.data);
        alert('Hvala! Vaša narudžba je uspješno poslana.');

        // Resetiraj narudžbu nakon slanja
        narucene_pizze.value = [];
    } catch (error) {
        console.error('Greška pri slanju narudžbe:', error);
        alert('Došlo je do greške pri slanju narudžbe. Molimo pokušajte ponovno.');
    }
}
```

Kako bismo bili sigurni da je narudžba pristigla, dodat ćemo `console.log` na Express poslužitelju na poletku `POST /narudzbe` endpointa:

```
// app/pizza-express/routes/narudzbe.js

router.post('/', (req, res) => {
    console.log('Primljeni podaci narudžbe:', req.body);
    // ostatak endpointa ...
});
```

Idemo testirati! Pošaljite narudžbu klikom na gumb **Naruči** unutar Vue.js aplikacije.



> Slika 24: Uspješno poslana narudžba unutar Vue.js aplikacije (prikaz u pregledniku)

Na Express poslužitelju, trebali bismo vidjeti ispisane podatke narudžbe u konzoli:

```
Primljeni podaci narudžbe: {
  narucene_pizze: [ { naziv: 'Al Tonno', velicina: 'jumbo', kolicina: 2 } ],
  podaci_dostava: {
    prezime: 'Pilić',
    adresa: 'Ilica 305, Zagreb',
    telefon: '091234567'
  }
}
```

Samostalni zadatak za Vježbu 3

Potrebno je implementirati Vue.js aplikaciju i Express poslužitelj prema uputama u ovoj skripti. Rezultat treba biti **funkcionalna Vue/Express aplikacija** za pregled dostupnih pizza, prikaz svih podataka o pizzama dohvaćenih s Expressa, te mogućnost slanja narudžbi na poslužitelj kao što je prikazano u primjerima.

Nakon što završite implementaciju, implementirajte sljedeće dodatne funkcionalnosti:

1. **Unos podataka za dostavu**: Umjesto hardkodiranih podataka za dostavu unutar `OrderFooter.vue` komponente, implementirajte obrazac koji omogućuje korisnicima da unesu svoje prezime, adresu i broj telefona prije slanja narudžbe. Validirajte unos kako biste osigurali da su svi potrebni podaci uneseni.
2. **Brisanje stavki iz košarice**: Dodajte mogućnost brisanja pojedinačnih stavki iz košarice unutar `OrderFooter.vue` komponente. Korisnici bi trebali moći ukloniti stavke prije nego što pošalju narudžbu.
3. **Prikaz statusa narudžbe**: Nakon slanja narudžbe, prikažite korisniku status narudžbe (npr. “Narudžba je uspješno poslana” ili “Došlo je do greške pri slanju narudžbe”) unutar same aplikacije, umjesto korištenja `alert` funkcija. **Poruku koju prikažete potrebno je dohvatiti s Express poslužitelja**!

4. Koristeći Vue Router, **implementirajte komponentu za pregled detalja o pojedinoj pizzi** na frontend adresi: `/pizze/:naziv`. Primjer: `/pizze/Margherita` bi trebao prikazati detalje o Margherita pizzi. Podaci koji se prikazuju trebaju biti dohvaćeni s Express poslužitelja koristeći definirani `GET /pizze/:naziv` endpoint. Korisnik se mora moći vratiti na popis svih pizza klikom na gumb “Natrag na popis”.